A Mini Project Report on

Reducing Hallucinations in Legal AI: Retrieval-Augmented Generation with a Gated Verification Chain

Submitted to Anurag University in partial fulfilment of the requirements for the award of the Degree of

Bachelor of Technology in Artificial Intelligence

Submitted by

Aleti Vivek Reddy — 23EG106C04

N. Akshith Sai — 23EG106C03

K. Uday — 23EG106C30

Under the Guidance of

S. Ravi Kumar

Assistant Professor

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

SCHOOL OF ENGINEERING

ANURAG UNIVERSITY

2023–2027

A Mini Project Report on

Reducing Hallucinations in Legal AI: Retrieval-Augmented Generation with a Gated Verification Chain

Submitted to Anurag University in partial fulfilment of the requirements for the award of the Degree of

Bachelor of Technology in Artificial Intelligence

Submitted by

Aleti Vivek Reddy — 23EG106C04

N. Akshith Sai — 23EG106C03

K. Uday — 23EG106C30

Under the Guidance of

S. Ravi Kumar

Assistant Professor

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

SCHOOL OF ENGINEERING

ANURAG UNIVERSITY

2023–2027

ANURAG UNIVERSITY

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

CERTIFICATE

This is to certify that the project report titled **Reducing Hallucinations in Legal AI: Retrieval-Augmented Generation with a Gated Verification Chain** is being submitted by **Aleti Vivek Reddy**, bearing **23EG106C04**, **N. Akshith Sai**, bearing **23EG106C03**, and **K. Uday**, bearing **23EG106C30**, in IV B.Tech I semester, Artificial Intelligence, and is a record of bonafide work carried out by them. The results embodied in this report have not been submitted to any other University for the award of any degree.

Student's Name	**Signature**
1. Aleti Vivek Reddy	1.
2. N. Akshith Sai	2.
3. K. Uday	3.

Guide	**Head of Department**	**External Examiner**
S. Ravi Kumar Assistant Professor	Dr. M. Trupti Professor	

DECLARATION

We hereby declare that the Mini Project report entitled **"Reducing Hallucinations in Legal AI: Retrieval-Augmented Generation with a Gated Verification Chain"**, submitted to the Department of Artificial Intelligence, Anurag University, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence, is a record of original work carried out by us under the guidance of **S. Ravi Kumar, Assistant Professor, Department of Artificial Intelligence**. The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma. Wherever the work of others, including datasets, pretrained models and publicly available code, has been used, it has been duly acknowledged and cited. The results reported are genuine and reproducible from the submitted code.

Use of AI tools: We declare that we have used an AI assistant (Claude) during this project to help with software development and with drafting and editing parts of this report. All experiments were run by us, and every result reported here was checked against the logged output of the system. We take full responsibility for the content of this report.

Place: Hyderabad

Date: _____

Name of the Student	**Roll No.**	**Signature**
Aleti Vivek Reddy	23EG106C04	
N. Akshith Sai	23EG106C03	
K. Uday	23EG106C30	

ACKNOWLEDGEMENT

We are grateful to Prof. Archana Mantri, Vice-Chancellor, Anurag University, for the facilities the University made available to us and for the environment in which this project could be carried out.

We thank Dr. V. Vijay Kumar, Dean, School of Engineering, for his encouragement throughout our course of study.

We sincerely thank Dr. M. Trupti, Professor and Head of the Department of Artificial Intelligence, for providing the resources and support we needed to complete this work.

We thank our project coordinator, Mr. Victor Daniel, Department of Artificial Intelligence, for organising the project reviews and keeping us on schedule.

We are especially indebted to our guide, S. Ravi Kumar, Assistant Professor, Department of Artificial Intelligence, Anurag University. His questions during the reviews shaped how we tested the system, and his feedback on each draft improved this report.

We also thank the developers of the open-source libraries this project is built on, in particular LanceDB, sentence-transformers, Hugging Face Transformers, LangChain, FastAPI and React, and the authors of the ContractNLI, CUAD and MAUD datasets.

An AI assistant (Claude) was used during this project to help with software development and with drafting and editing parts of this report. All experiments were run by us, and every result reported here was checked against the logged output of the system.

Aleti Vivek Reddy (23EG106C04)

N. Akshith Sai (23EG106C03)

K. Uday (23EG106C30)

ABSTRACT

Large language models answer legal questions fluently, but they sometimes cite provisions or cases that do not exist, or state what a clause does not say. In legal work an unsupported answer is worse than no answer, because the reader may rely on it. Retrieval-augmented generation (RAG) reduces this problem by giving the model passages from trusted documents, but it does not check the answer the model finally writes.

This project builds Legal AI, a question-answering system for statutes and contracts in which every answer is checked against its sources before the user sees it. Documents are split with summary-augmented chunking (SAC) when they have Act, Chapter, Section and Clause structure, and by paragraph otherwise. The chunks are stored in an embedded LanceDB index that holds both dense vectors and a BM25 keyword index; retrieval merges the two with reciprocal rank fusion and reorders the result with a cross-encoder. The language model must end every claim with the identifier of the chunk that supports it, and a six-layer verification chain (V1–V6) checks each claim for citation existence, entailment by a natural language inference (NLI) model, atomic-fact fidelity and, optionally, self-consistency. The checks are combined into a Verification Confidence Score (VCS); the answer is shown only if the VCS reaches 0.60, together with a proof listing the quoted evidence and each check's verdict. The system is used through a React web application backed by FastAPI, and its pipeline is composed with LangChain.

On ten labelled questions over five statutes, SAC raised retrieval F1 from 0.333 to 0.417 compared with paragraph chunking. Of 28 deliberately corrupted claims, the full chain showed none to the user; removing only the score gate showed all 28. In an end-to-end test through the web interface with five real documents and fifteen questions whose answers were written down in advance, no wrong answer was ever shown as verified. The main weakness is false abstention: the system refuses some answerable questions, which makes calibration of the gate the most important next step.

Keywords: Legal AI, Retrieval-Augmented Generation, Hallucination, Citation Verification, Natural Language Inference, Hybrid Search, LangChain, Abstention.

TABLE OF CONTENTS

Certificate i

Declaration ii

Acknowledgement iii

Abstract iv

List of Figures vii

List of Tables viii

List of Abbreviations ix

CHAPTER 1 INTRODUCTION 1

1.1 Background / Overview 1

1.2 Motivation 2

1.3 Problem Statement 3

1.4 Objectives 3

1.5 Scope and Limitations 4

1.6 Organization of the Report 5

CHAPTER 2 LITERATURE SURVEY 6

2.1 Review of Related Work 6

2.2 Summary Table 9

2.3 Research Gap Identified 10

CHAPTER 3 SYSTEM ANALYSIS AND REQUIREMENTS 12

3.1 Existing Approaches and their Drawbacks 12

3.2 Feasibility Study 12

3.3 Hardware Requirements 13

3.4 Software Requirements 14

3.5 Client (User) Requirements 14

CHAPTER 4 DATASET, DESIGN AND METHODOLOGY 16

4.1 Overall Pipeline and System Architecture 16

4.2 Datasets 17

4.3 Document Ingestion and Adaptive Chunking 18

4.4 Hybrid Indexing and Retrieval 18

4.5 Citation-Forced Generation 19
4.6 The Verification Chain (V1–V6) 19
4.7 Abstention and General-Knowledge Answers 21
4.8 Data Flow and UML Diagrams 21
4.9 Module Design and Organisation 24
4.10 Baselines and Comparison Methods 25
CHAPTER 5 IMPLEMENTATION 27
5.1 Technologies Used 27
5.2 Pipeline Configuration 27
5.3 Procedures 28
5.4 Key Code Snippets 30
5.5 Web Application and Deployment 31
5.6 The LangChain Version 33
5.7 Testing and Validation 33
CHAPTER 6 RESULTS AND DISCUSSION 36
6.1 Output of the Working System 36
6.2 Retrieval and Chunking 39
6.3 Verification Ablation 40
6.4 Error Analysis: False Abstentions 42
6.5 Batch Test on Real Contracts 42
6.6 End-to-End Test Through the Interface 42
6.7 LangChain Version Compared with Plain Python 44
6.8 Ethical and Responsible AI Considerations 45
CHAPTER 7 CONCLUSION AND FUTURE SCOPE 46
7.1 Conclusion 46
7.2 Limitations 46
7.3 Future Enhancements 47
REFERENCES 49
APPENDICES 51

LIST OF FIGURES

Fig. No.	Title	Page No.
1.1	High-level block diagram of Legal AI: retrieval, citation-forced generation, verification and the gate that decides whether an answer is shown	2
4.1	System architecture: web interface, FastAPI backend, pipeline modules, the embedded index and the models	17
4.2	The verification chain. V1–V4 check each claim, V5 turns the scores into one decision and V6 records the evidence	21
4.3	Data flow diagram, level 0 (context diagram)	22
4.4	Data flow diagram, level 1	22
4.5	Use case diagram	23
4.6	Sequence diagram for answering a question	24
5.1	Home page: index size and the three stages of the system	32
5.2	Ingest page: drag-and-drop upload of a PDF or text file	33
6.1	Uploading the same document twice: the duplicate is refused and can be replaced	36
6.2	Ask page: a verified answer with the proof of the first claim open, showing the quoted passage and the verdict of each check	37
6.3	An abstention: the indexed documents do not cover the question, so nothing is asserted	37
6.4	A paraphrased question (“thrown out by force”) answered from the eviction clause at VCS 1.00	38
6.5	Search page: the same question through dense, keyword and hybrid retrieval	39
6.6	Evaluation page: the embedding map of all chunks and the cards of the logged experiments	39
6.7	Retrieval F1 for SAC and paragraph chunking under each retrieval setting	40
6.8	Corrupted claims shown to the user when each layer is removed (28 in total)	42
6.9	A verified but incomplete answer: both claims are correct, but one of the three agreements asked about is missing	44

LIST OF TABLES

Table No.	Title	Page No.
2.1	Summary of related work and how this project builds on it	9
3.1	Software requirements and development environment	14
4.1	Datasets used in the project	17
4.2	Modules of the system	24
5.1	Pipeline configuration used in the experiments	27
5.2	Automated test suites	33
5.3	End-to-end test scenarios	34
6.1	Retrieval results over 10 questions (k = 5, n = 50)	39
6.2	Verification ablation (clean set: 10 questions, 14 claims; bad claims: 28)	41
6.3	End-to-end test results (15 questions)	43
6.4	Acceptance checks for the LangChain version	44

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
BM25	Best Match 25 (Okapi keyword ranking function)
CPU	Central Processing Unit
CUAD	Contract Understanding Atticus Dataset
DFD	Data Flow Diagram
FTS	Full-Text Search
GPU	Graphics Processing Unit
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LCEL	LangChain Expression Language
LLM	Large Language Model
MAUD	Merger Agreement Understanding Dataset
NDA	Non-Disclosure Agreement
NLI	Natural Language Inference
PCA	Principal Component Analysis
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RRF	Reciprocal Rank Fusion
SAC	Summary-Augmented Chunking
UI	User Interface
UML	Unified Modeling Language
VCS	Verification Confidence Score

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND / OVERVIEW

Legal questions are answered from sources. Whether a tenant can be evicted, how long a confidentiality obligation lasts or which court hears an appeal depends on the exact words of a statute or contract, and anyone relying on an answer needs to be able to find those words. A legal answer is useful only when it can be traced back to the provision behind it. This is what separates legal question answering from most other applications of language technology: in casual use, a fluent and mostly-correct answer is acceptable, but in legal work the reader may act on the answer, and an unsupported statement can cause real harm.

Large language models (LLMs) such as GPT, Gemini and Claude have made it easy to ask legal questions in plain language, and their answers are fluent and confident. The difficulty is that fluency is not accuracy. These models are known to hallucinate: they can produce statements that sound authoritative but are not supported by any source. In 2023 a United States federal court sanctioned two lawyers in Mata v. Avianca, Inc. after they filed a brief citing court decisions that a chatbot had invented [1]. A later study of commercial legal research tools found that even products built on retrieval still produce hallucinated answers at a noticeable rate [2].

Retrieval-augmented generation (RAG) [3] is the standard remedy. Instead of answering from memory, the model is given passages retrieved from a trusted document collection and asked to answer from them. RAG improves grounding, but it does not guarantee it: the model can still misread a passage, combine two passages incorrectly, or cite a passage that does not support what it wrote. Nothing in a basic RAG system checks the final answer before a user reads it.

This project builds **Legal AI**, a question-answering system for statutes and contracts in which verification is a required pipeline stage rather than an optional report. Documents uploaded by the user are chunked in a structure-aware way, indexed in an embedded hybrid (vector plus keyword) database, and retrieved with rank fusion and cross-encoder reranking. The language model is forced to cite, for every claim it makes, the

identifier of the chunk that supports it, and a six-layer verification chain checks every claim before the answer is shown. When the evidence is not good enough, the system abstains and says so. Figure 1.1 shows this flow at the highest level: the answer reaches the user only after it has passed through retrieval, citation-forced generation, verification and a gate.

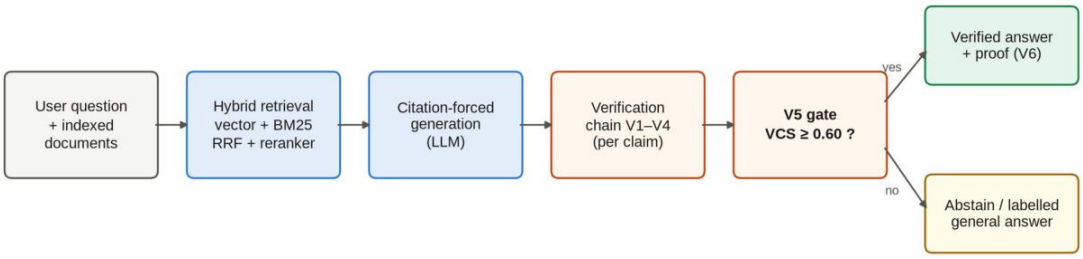


Fig. 1.1 High-level block diagram of Legal AI: retrieval, citation-forced generation, verification and the gate that decides whether an answer is shown.

1.2 MOTIVATION

The motivation for this project was not to build another legal chatbot, but a system that treats verification as a required step: an answer should reach the user only after each of its claims has been checked against the passage it cites, and when the check fails, the system should say so instead of guessing.

Three observations shaped this goal. First, the consequences of a wrong legal answer are asymmetric. A correct answer saves the reader some minutes of searching; a wrong answer that the reader trusts can lead to a missed deadline, a wrong filing or a sanction, as the Mata v. Avianca episode showed [1]. A system that is sometimes silent but never confidently wrong is therefore more useful in this domain than one that always answers.

Second, the parts needed to check an answer already exist as published research — natural language inference models can test whether a passage entails a claim [15], atomic-fact decomposition can test long answers piece by piece [17], and self-consistency sampling can flag unstable claims [18] — but they are almost always used as offline evaluation tools. What is missing in deployed systems is a component that runs these checks at answer time and acts on the result.

Third, the evidence shown to the user matters as much as the checking itself. A verdict of “verified” is only as trustworthy as the process behind it, so the system should

expose, for every claim, the exact passage it cited, the verdict of every check and the score that resulted. The reader can then audit the answer instead of trusting it.

1.3 PROBLEM STATEMENT

Existing legal question-answering systems built on RAG ground their answers in retrieved documents but do not verify the answer before showing it, do not show the user the evidence behind each statement, and rarely refuse when the evidence is weak. In particular, the project addresses the following problems:

1. **Unverified answers:** RAG systems show the model's answer as it is, even when a claim is not supported by the retrieved text or cites a passage that was never retrieved.
2. **Loss of legal context in chunking:** fixed-size chunking cuts a clause away from the section that gives it meaning, so a phrase such as “within thirty days” is retrieved without the rule it belongs to.
3. **No evidence for the reader:** users are rarely shown which passage supports which sentence, so they cannot check an answer without reading the whole document.
4. **Guessing instead of refusing:** when the documents do not contain an answer, systems tend to answer anyway rather than state that the answer is not available.

Hence there is a need for a legal question-answering system that (1) retrieves relevant passages from statutes and contracts using structure-aware chunking and hybrid search, (2) forces the model to cite a passage for every claim, (3) checks every claim through a chain of complementary verification layers combined into a single confidence score, and (4) shows an answer only when that score is high enough, together with an inspectable proof, and refuses otherwise.

1.4 OBJECTIVES

- Build an ingestion pipeline for PDF and text documents that detects statute structure, applies summary-augmented chunking where possible and paragraph chunking otherwise, and rejects duplicate uploads.
- Implement hybrid retrieval (dense vectors and BM25) with rank fusion and cross-encoder reranking over a single embedded index.

- Implement citation-forced generation that works with several LLM providers.
- Design and implement the V1–V6 verification chain, the Verification Confidence Score and the abstention gate.
- Provide a web interface in which every answer carries a verdict, a score and a proof.
- Evaluate retrieval and verification with ablation experiments and an end-to-end test on real documents, and report the results honestly, including failures.

1.5 SCOPE AND LIMITATIONS

The system answers questions in English over documents that the user uploads as PDF or text files: statutes with a section structure and contracts such as non-disclosure and commercial agreements. It is a research prototype and does not give legal advice. Possible uses include:

- **Law students and legal education:** asking questions about a statute and seeing the exact clause each answer comes from.
- **Contract review:** checking terms such as governing law, confidentiality periods or remedies across a set of agreements, with each answer linked to its paragraph.
- **Legal aid and small firms:** a first reading of documents where every statement can be verified quickly, and where missing information is reported rather than invented.
- **Research on trustworthy AI:** the verification chain and its logged experiments can be reused to study hallucination detection in other domains.

The scope is deliberately limited in several ways. The system answers from the documents the user has uploaded, not from the whole body of law; it does not perform legal reasoning across documents beyond what the retrieved passages state; and the verification chain checks what an answer says, not what it leaves out, so a factually correct but incomplete answer can still be shown (this limitation is examined in Chapter 6). The evaluation is small and controlled rather than large and statistical: ten labelled questions over five synthetic statutes, a batch test on ten real agreements and a fifteen-question end-to-end test. The numbers reported are observations from logged runs, not population estimates.

1.6 ORGANIZATION OF THE REPORT

Chapter 2 reviews the literature this project builds on — retrieval-augmented generation, chunking for legal text, hybrid retrieval, natural language inference and hallucination detection — and identifies the research gap. Chapter 3 analyses the shortcomings of existing systems, examines feasibility, and lists the hardware and software requirements. Chapter 4 describes the datasets, the overall design (data flow and UML diagrams, module organisation) and the methodology of every pipeline stage, including the verification chain and its scoring equations. Chapter 5 covers the implementation: the technologies used, the build procedure, key code, the web application and the testing strategy. Chapter 6 presents the outputs and results: retrieval and verification ablations, the batch test on real contracts, the end-to-end test through the interface, error analysis and the ethical considerations. Chapter 7 concludes and lists future enhancements. The appendices contain the repository guide, the REST API reference, a sample request and response, and a troubleshooting guide.

CHAPTER 2

LITERATURE SURVEY

2.1 REVIEW OF RELATED WORK

2.1.1 Retrieval-Augmented Generation

Lewis et al. [3] introduced retrieval-augmented generation (RAG), in which a retriever selects passages from a document collection and a generator conditions its answer on them. The original formulation combined a dense passage retriever with a sequence-to-sequence generator and showed that grounding generation in retrieved evidence improves factuality on knowledge-intensive tasks. Most current legal assistants follow this design: documents are split into chunks, each chunk is embedded as a vector, the chunks closest to the question are retrieved, and an LLM writes the answer. The architecture has two well-known failure points that this project targets: the retriever can return the wrong passages, and the generator can depart from the passages it was given. RAG constrains what the model sees, but nothing in the basic design constrains what the model finally says.

2.1.2 Chunking for Legal Text

How documents are split into retrievable units matters more in law than in most domains, because legal meaning is hierarchical: a clause depends on its section, the section on its chapter, and the chapter on the Act. The simplest systems cut documents into fixed-size windows. Reuter et al. [4] showed that in large legal collections this often retrieves text from the wrong document entirely, and proposed summary-augmented chunking (SAC), in which each chunk is prefixed with a summary so that it keeps its document-level context. In their method the summary is written by an LLM for the whole document, which adds a model call to every ingestion. This project adopts the idea but replaces the generated summary with an extractive one built from the section heading and its first sentence, so that ingestion needs no model call at all (Section 4.3).

2.1.3 Dense Retrieval, Keyword Retrieval and Fusion

Dense retrieval represents questions and passages as vectors and ranks passages by similarity. Reimers and Gurevych [12] introduced Sentence-BERT, which fine-tunes BERT in a siamese configuration to produce sentence embeddings that can be compared with cosine similarity, making dense semantic search practical at scale; the all-MiniLM-L6-v2 model used in this project is a compact descendant of that line of work. Dense retrieval finds passages with similar meaning even when the wording differs, which matters for questions phrased in everyday language about documents phrased in legal language.

Keyword retrieval, typified by BM25, ranks passages by weighted term overlap. It cannot generalise across synonyms, but it is exact where dense retrieval is fuzzy: section numbers, party names and terms of art are found reliably. Because the two methods fail in different places, combining them is standard practice. Cormack et al. [13] showed that reciprocal rank fusion (RRF), which scores each document by the reciprocal of its rank in each result list, outperforms more elaborate rank-learning methods while needing no tuning. Cross-encoder rerankers such as the BGE reranker of Xiao et al. [14] then read the question and each candidate passage together through one transformer, giving a more precise relevance score than any bi-encoder, at a higher computational cost. The standard architecture, which this project follows, is therefore: a cheap first stage with high recall (dense plus keyword, RRF-fused), followed by an expensive second stage with high precision (cross-encoder reranking) over a large candidate pool.

2.1.4 Natural Language Inference

Natural language inference (NLI) is the task of deciding whether a premise entails, contradicts or is neutral towards a hypothesis. Williams et al. [16] built MultiNLI, a broad-coverage corpus of premise–hypothesis pairs across many genres, which became the standard training set for the task. Liu et al. [15] showed with RoBERTa that careful pretraining substantially improves such classification tasks; the roberta-large-mnli model, RoBERTa fine-tuned on MultiNLI, is the entailment checker used in this project. Framing answer verification as NLI is natural: the cited passage is the premise, the claim is the hypothesis, and a claim is supported only if the passage entails it. The limitation, which Chapter 6 measures, is that a general-domain NLI model is conservative on legal text: it returns NEUTRAL on some claims a lawyer would accept,

because legal entailment often relies on document context the premise sentence does not repeat.

2.1.5 Hallucination Detection

Min et al. [17] proposed FActScore, which decomposes a long generated text into atomic facts and verifies each one against a knowledge source, so that one wrong clause in an otherwise correct sentence is not averaged away. Manakul et al. [18] proposed SelfCheckGPT, a black-box method that samples the model several times and treats statements that do not recur across samples as likely hallucinations, on the reasoning that a model hallucinating has no stable source to return to. Niu et al. [7] built RAGTruth, a corpus of RAG outputs with word-level hallucination annotations, for training and benchmarking hallucination detectors. All three treat detection as measurement: they produce scores and benchmarks, but none of them decides, inside a running system, whether an answer may be shown. In this project FActScore's decomposition idea becomes check V3, SelfCheckGPT's sampling idea becomes the optional check V4, and the decision rule the literature leaves open becomes the V5 gate.

2.1.6 Legal RAG Systems and Studies

The legal domain has both the clearest need for verification and the clearest evidence of its absence. Mata v. Avianca [1] is the best-known cautionary case: a chatbot invented court decisions, the lawyers filed them, and the court sanctioned the lawyers. Magesh et al. [2] systematically assessed commercial AI legal research tools and found that hallucinations persist even in products that advertise retrieval grounding — evidence that retrieval alone does not close the gap. Figueiredo et al. [5] describe a production legal RAG system for Brazilian Portuguese with a multi-stage anti-hallucination pipeline, including an audit of legal references after generation; their audit focuses on whether cited authorities exist and on rewriting unsupported text, whereas in this project citation existence is only the first of six checks and a failed check leads to withholding, not rewriting. Maghakian et al. [6] take the opposite architectural route and move retrieval inside the LLM itself (embedding-free RAG), which removes the chunking problem but raises the cost of every query; this project keeps retrieval outside the model, where it is cheap and inspectable. Pipitone and Alami [8] built LegalBench-RAG, a benchmark for retrieval quality in legal RAG, which this project lists as future evaluation work.

2.1.7 Legal Datasets

Three expert-annotated contract datasets are used in this project as sources of real documents. Koreeda and Manning [9] built ContractNLI, which frames contract review as document-level NLI over non-disclosure agreements: for each agreement and hypothesis, the dataset records whether the agreement entails, contradicts or does not mention the hypothesis. Hendrycks et al. [10] built CUAD, a corpus of commercial contracts annotated by lawyers with the clauses relevant to 41 review categories, designed to measure clause extraction. Wang et al. [11] built MAUD, an expert-annotated dataset of merger agreements for legal-standard understanding. In this project these corpora are not used to train anything; they supply realistic, held-out documents for the batch and end-to-end tests in Chapter 6 — documents with none of the statute structure the chunker was designed around, which is exactly what makes them a fair stress test.

2.2 SUMMARY TABLE

Table 2.1 Summary of related work and how this project builds on it

Work	**Approach**	**Limitation**	**In this project**
Lewis et al. [3], 2020, NeurIPS	RAG: retrieve, then generate	No check on the final answer	Every claim is verified before display
Reuter et al. [4], 2025, NLLP	Summary-augmented chunking	Summary written by an LLM for the whole document	Extractive, section-level summary; no model call at ingestion
Figueiredo et al. [5], 2026, PROPOR	Citation audit after generation	Focus on citation existence and rewriting	Existence is only the first of six checks, combined into one score
Maghakian et al. [6], 2025, EMNLP Findings	Retrieval inside the LLM	High cost per query	Cheap embedded hybrid retrieval plus reranking
Min et al. [17], 2023, EMNLP (FActScore)	Atomic-fact checking	Evaluation tool, not a gate	Used as check V3 inside a gate
Manakul et al. [18], 2023, EMNLP (SelfCheckGPT)	Consistency across samples	Extra model calls	Optional check V4, weighted in the score

Work	Approach	Limitation	In this project
Niu et al. [7], 2024, ACL (RAGTruth); Pipitone & Alami [8], 2024 (LegalBench-RAG)	Benchmarks	Measure hallucination, do not prevent it	Prevention through the VCS gate
Cormack et al. [13], 2009, SIGIR	Reciprocal rank fusion	Fusion only; no precision stage	RRF fused pool feeds a cross-encoder reranker [14]
Reimers & Gurevych [12], 2019, EMNLP	Sentence embeddings (bi-encoder)	Misses exact legal wording	Combined with BM25 keyword search
Liu et al. [15] on MultiNLI [16]	NLI entailment classification	General-domain; conservative on legal text	Used as check V2; limitation measured in Chapter 6

2.3 RESEARCH GAP IDENTIFIED

Across this literature, detection and prevention remain separate. Retrieval research improves what the model reads; hallucination research measures what the model writes; legal benchmarks score both. What is missing is the connective tissue — a deployed decision rule that turns detector outputs into “show” or “withhold” on every answer. Specifically, the gaps this project addresses are:

- A gate that combines several complementary checks into one score and acts on it, rather than detectors that only report.
- A per-claim proof that shows the user the exact passage, the verdict of every check and the claim's contribution to the score.
- Structure-aware chunking for statutes that keeps section context without calling a model for every document, with a fallback for documents that have no structure.
- Honest refusal: a system that says it cannot answer when the evidence is insufficient, while still being able to answer general legal-concept questions, clearly labelled as unverified.

- Measured evidence of what each part contributes, through ablations and tests on real documents, rather than a single headline accuracy figure.

CHAPTER 3

SYSTEM ANALYSIS AND REQUIREMENTS

3.1 EXISTING APPROACHES AND THEIR DRAWBACKS

The existing systems reviewed in Chapter 2 share a common pipeline — chunk, embed, retrieve, generate — and a common set of weaknesses when applied to legal work:

- **Answers are shown unchecked.** Basic RAG trusts the model to stay faithful to the passages it was given; nothing prevents an unsupported sentence from reaching the user.
- **Citations can be invented.** Models can cite a document or passage that was never retrieved, and many interfaces display such citations as if they were real.
- **Context is lost in chunking.** Fixed-size chunks separate clauses from their section headings, and document-level summaries require an extra LLM call for every document.
- **Detectors work in isolation.** Individual hallucination detectors report a score, but there is rarely a single decision rule that turns their scores into “show” or “withhold”.
- **Little transparency.** Users are not shown the quoted evidence and the result of each check, so they must either trust the answer or read the whole document.
- **Heavy infrastructure.** Many systems require a separate vector database server and are tied to one model provider.

The proposed system answers each drawback directly: a verification chain with a gate (unchecked answers, isolated detectors), a citation-existence check against the retrieved set (invented citations), summary-augmented chunking with an extractive summary (lost context, ingestion cost), a per-claim proof object (transparency), and an embedded database with provider adapters (infrastructure).

3.2 FEASIBILITY STUDY

3.2.1 Technical Feasibility

Every component the design needs is available as mature open-source software or as a hosted API. Embeddings, reranking and entailment run locally on a CPU with models of modest size (the largest, roberta-large-mnli, is about 1.4 GB). LanceDB stores vectors and a BM25 full-text index in one on-disk table inside the application process, so no database server has to be installed or administered. The only external dependency is the LLM API, and the generation stage is written behind a provider adapter so that Groq, Gemini, OpenAI or Anthropic models can be used interchangeably. The riskiest technical assumption — that a general-purpose NLI model can judge legal entailment well enough to be useful — was tested early on synthetic statutes and monitored throughout; it works, with a conservatism cost that is measured rather than hidden (Section 6.4).

3.2.2 Economic Feasibility

The project was built and evaluated at essentially zero monetary cost. All local models are free to download and run; the development machines were ordinary laptops without GPUs; and the LLM calls used free tiers (Groq for the web interface, Gemini for ablation runs). The free tiers impose rate limits that interrupted some batch runs — an operational cost paid in time rather than money, discussed in Section 6.5 — but no component of the system requires a paid service to operate. A deployment at larger scale would pay only for LLM tokens and ordinary hosting.

3.2.3 Operational Feasibility

The system runs on a standard Windows, Linux or macOS machine with one script: start.bat opens the backend and the web interface together, and the first start downloads the local models once. Users interact through a browser with five self-explanatory pages, and every answer explains itself through its badge, score and proof, so no training in the system's internals is needed to judge an answer. The index, the chat history and the experiment logs are plain local files, which makes backup and inspection trivial.

3.3 HARDWARE REQUIREMENTS

- **Minimum:** a dual-core CPU, 8 GB RAM and about 5 GB of free disk space for the models, the index and the Python environments.

- **Recommended:** a quad-core CPU and 16 GB RAM. The NLI model (roberta-large-mnli) is the largest local model, at about 1.4 GB, and reranking runs noticeably faster on a better CPU.
- **GPU:** not required; all local models run on the CPU. A GPU shortens reranking and NLI time but is optional.
- **Network:** an internet connection for the LLM API and for the one-time download of the local models.

All development and all experiments reported in Chapter 6 were run on laptop CPUs, which is why reranking latency (6.7 to 12.8 seconds per question on a laptop CPU) is reported as a real cost of the design rather than hidden behind server hardware.

3.4 SOFTWARE REQUIREMENTS

Table 3.1 Software requirements and development environment

Layer	**Software**
Operating system	Windows 10/11, Linux or macOS
Language runtimes	Python 3.11 (backend, pipeline, experiments, tests); Node.js with npm (web interface)
Backend framework	FastAPI with Uvicorn; Pydantic for request/response models
Pipeline composition	LangChain (langchain, langchain-community, langchain-huggingface, langchain-groq and the other provider packages); LCEL for chain composition
Retrieval and models	LanceDB; sentence-transformers (all-MiniLM-L6-v2, BAAI/bge-reranker-base); Hugging Face Transformers with PyTorch (roberta-large-mnli); pypdf; rank-bm25
LLM access	An API key for at least one provider: Groq, Google Gemini, OpenAI or Anthropic
Frontend	React 18, React Router, Vite, Tailwind CSS, Framer Motion
Testing and tools	pytest; Playwright with Chromium (end-to-end test); Matplotlib (result charts); Git and GitHub; a modern web browser

3.5 CLIENT (USER) REQUIREMENTS

From the user's point of view, the system must:

- Upload statutes and contracts as PDF or plain-text files, and replace or delete them later.

- Ask questions in plain English and receive answers in which every sentence is linked to its source passage.
- See clearly whether an answer is verified, withheld or general knowledge, and inspect the proof of any claim.
- Keep a history of conversations that survives a page reload.
- Compare retrieval methods and view evaluation results without reading code.

CHAPTER 4

DATASET, DESIGN AND METHODOLOGY

4.1 OVERALL PIPELINE AND SYSTEM ARCHITECTURE

Legal AI is organised as a web application on top of a question-answering pipeline. The React interface talks to a FastAPI backend over HTTP; the backend runs the pipeline stages, each implemented as a separate module, and stores documents in an embedded LanceDB index on disk. The language model is called through a provider adapter, while the embedding, reranking and NLI models run locally. Figure 4.1 shows the components and how they connect.

The pipeline has two paths. The ingestion path runs when a document is uploaded: the file is de-duplicated, cleaned, chunked and embedded, and its chunks are written to the index. The question path runs for every query: candidates are retrieved from the index and reranked, the best chunks are given to the language model under citation-forcing rules, the model's claims are verified layer by layer, and the gate decides whether the user sees a verified answer, an abstention or a clearly-labelled general-knowledge reply. Every stage passes plain data to the next — chunk records, claims, verdicts — which keeps each stage testable on its own.

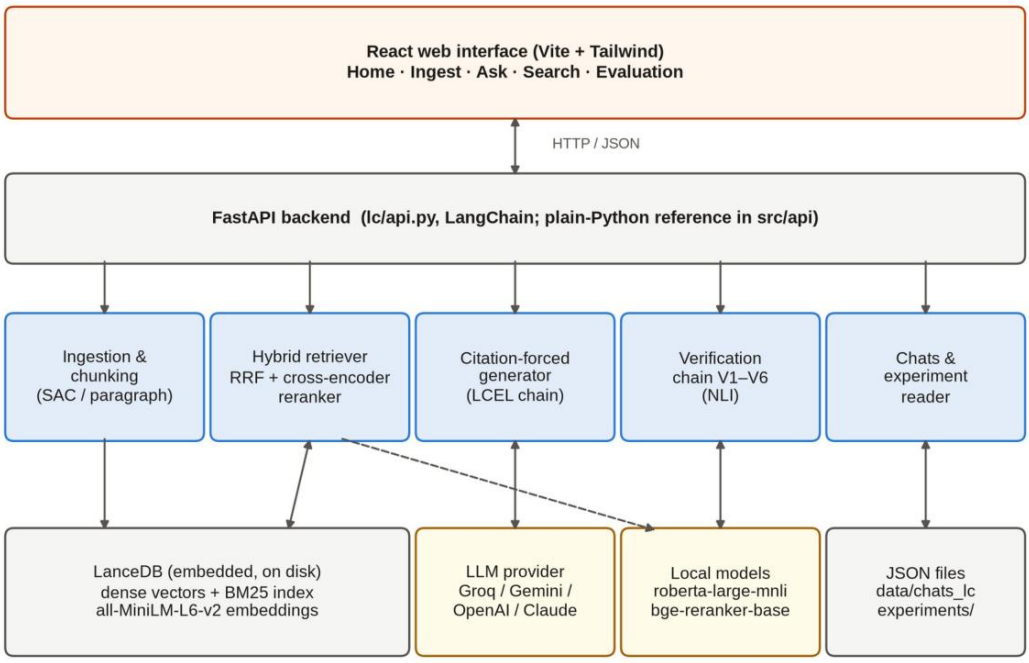


Fig. 4.1 System architecture: web interface, FastAPI backend, pipeline modules, the embedded index and the models.

4.2 DATASETS

Two kinds of data are used: synthetic statutes written by the team for controlled experiments, and real contracts from three expert-annotated public datasets for realistic testing. Table 4.1 summarises them.

Table 4.1 Datasets used in the project

Data	**Type**	**Size**	**Used for**
Synthetic statutes (written by the team)	Statutes with Act / Chapter / Section / Clause structure: urban tenancy, data privacy, consumer protection, small-business licensing, waste management	5 Acts; 10 labelled questions; 14 gold clauses	Retrieval and verification ablations
ContractNLI [9]	Non-disclosure agreements (PDF)	10 in the batch test; 3 in the end-to-end test	Batch test, end-to-end test
CUAD [10]	Commercial contracts	1 in the end-to-end test	End-to-end test
MAUD [11]	Merger agreements	Used during development testing	Behaviour on long real contracts
End-to-end question set	Questions with expected answers written in advance	15 questions, 6 categories	End-to-end test (Section 6.5)

Writing the five statutes ourselves meant the correct answer to every question was known exactly: each of the ten labelled questions names the clause (or clauses) that answers it, fourteen gold clauses in all. The real documents were taken from ContractNLI, CUAD and MAUD under their published licences. These are contracts without statute structure, so they exercise the paragraph fallback and show how the system behaves on documents it was not designed around.

Splits and leakage. No model is trained or fine-tuned in this project; every model is used as published. There is therefore no train/validation/test split in the usual sense, and no risk of training-set leakage. The equivalent discipline here is evaluation hygiene: the labelled questions and gold clauses were fixed before the ablation runs, the fifteen end-

to-end questions and their expected answers were committed to the repository before any document was uploaded, and every reported number is read from a logged results file in the experiments/ folder rather than from memory.

4.3 DOCUMENT INGESTION AND ADAPTIVE CHUNKING

An uploaded file is first fingerprinted with SHA-256. If the same content, or a file with the same name, is already indexed, the upload is refused so the index never holds a document twice. Text is extracted with pypdf (for PDFs) or read directly, and whitespace is normalised without disturbing headings.

A line-by-line parser then looks for statute structure: an Act title, chapters, numbered sections and clauses. When it finds this structure, the document is split with summary-augmented chunking: one chunk for each section body and one for each clause. Every chunk carries a short contextual summary made of the section heading and its first sentence, and this summary is embedded together with the chunk text. A clause such as "within thirty days" is therefore found together with the rule it belongs to. Unlike the original SAC method [4], which asks an LLM to summarise the whole document, the summary here is extractive, so ingestion needs no model call.

Most contracts have no such structure. For them the system falls back to paragraph chunking: the text is split at blank lines, and paragraphs longer than 700 characters are split into sentences and packed into windows of at most 700 characters. The chunk identifiers show which method was used: urban_tenancy_act_2019::s7:a means section 7, clause (a) of that Act, and NDA_7::p21 means paragraph 21 of that agreement. These identifiers do double duty: they address chunks in the index, and they are the citation tokens the language model must emit, so a citation can be resolved to its exact source text without ambiguity.

4.4 HYBRID INDEXING AND RETRIEVAL

Chunks are embedded with the all-MiniLM-L6-v2 sentence-transformer (384 dimensions, normalised) and stored in LanceDB, an embedded database that keeps the vectors and a BM25 full-text index in one on-disk table, so no database server is needed. A question is searched in both indexes and the two rankings are merged with reciprocal rank fusion (RRF):

$$\text{score}(d) = \Sigma\ 1 / (60 + \text{rank}_i(d)) \quad (4.1)$$

where $\text{rank}_i(d)$ is the position of chunk d in the i-th ranking. A chunk that ranks well in both the dense and the keyword list scores higher than one found by only one of them. The constant 60 is the standard RRF damping value from Cormack et al. [13]; it prevents the very top of either list from dominating the fusion.

The fused candidates are then rescored by the BAAI/bge-reranker-base cross-encoder, which reads the question and each passage together, and the top k are kept (k = 12 in the web interface). The candidate pool is deliberately much larger than k (at least 100), because the reranker can only promote passages that the first stage returned. This pool size is not a cosmetic choice: Section 5.3.4 describes how a pool that was too small caused silent failures on real contracts, and how the defect was found and fixed.

4.5 CITATION-FORCED GENERATION

The retrieved chunks are placed in a prompt, each labelled with its identifier. The rules given to the model are strict: use only the supplied passages, write one claim per line, end every line with the identifier of the chunk that supports it, and if the passages cannot answer the question, reply with a line starting with INSUFFICIENT_CONTEXT. A parser then turns the reply into a list of claims, each with the chunk identifiers it cites. The strict one-claim-per-line, citation-terminated format is what makes verification tractable: the parser can map each claim back to its cited evidence deterministically, with no natural-language guessing about which sentence rests on which source.

The model is reached through a small adapter, so Groq, Gemini, OpenAI or Claude models can be used by changing one environment variable; the web interface uses the openai/gpt-oss-120b model served by Groq. The adapter isolates the pipeline from provider differences in message formats and error behaviour, and it is the only stage of the pipeline that leaves the machine.

4.6 THE VERIFICATION CHAIN (V1–V6)

Every answer passes through six layers in order (Figure 4.2):

- **V1 – Citation existence:** every cited identifier must belong to a chunk that was actually retrieved. A citation to anything else is treated as fabricated.

- **V2 – Entailment:** the NLI model roberta-large-mnli [15] reads the cited passage as the premise and the claim as the hypothesis, and returns ENTAILS, NEUTRAL or CONTRADICTS.
- **V3 – Atomic fidelity:** the claim is split at semicolons, conjunctions and sentence breaks into atomic facts, each fact is checked with the NLI model, and the fraction entailed is the fidelity.
- **V4 – Self-consistency (optional):** the answer is generated again several times and the claim's recurrence rate is measured; a claim that does not come back is suspicious.
- **V5 – Score and gate:** the per-claim checks are combined into the Verification Confidence Score and compared with the threshold.
- **V6 – Proof object:** for every claim the system records the cited chunks, the quoted passage, each layer's verdict and the claim's contribution to the score.

A failed V1 check or a CONTRADICTS verdict from V2 sets a claim's score to zero — these are hard gates that no weighted average can overcome. Otherwise the claim score is a weighted mean of the layers that ran:

$$s = (0.40 \cdot e + 0.35 \cdot f + 0.25 \cdot c) \,/\, (\text{sum of the weights of the layers that ran}) \quad (4.2)$$

where e = 1 if V2 returns ENTAILS and 0 otherwise, f is the V3 fidelity and c the V4 recurrence rate. A layer that did not run is removed from both the numerator and the denominator, so skipping V4 never lowers a score. The answer's score is the mean over its N claims, and the answer is shown only if it reaches the threshold $\tau = 0.60$:

$$VCS = (1/N)\, \Sigma\, s_i; \;\; \text{show the answer if } VCS \geq 0.60, \text{ otherwise abstain} \quad (4.3)$$

The weights (0.40 for entailment, 0.35 for fidelity, 0.25 for consistency) order the checks by how directly each one tests support: whole-claim entailment is the strongest signal, atomic fidelity catches partial errors inside otherwise-entailed claims, and consistency is a weaker, indirect signal. The threshold of 0.60 is a default; Chapter 6 shows that it has not been calibrated and that calibrating it is the most important piece of future work. Each layer can be switched off independently, which is what makes the ablation study in Section 6.3 possible: the same code path runs with one layer removed and everything else unchanged.

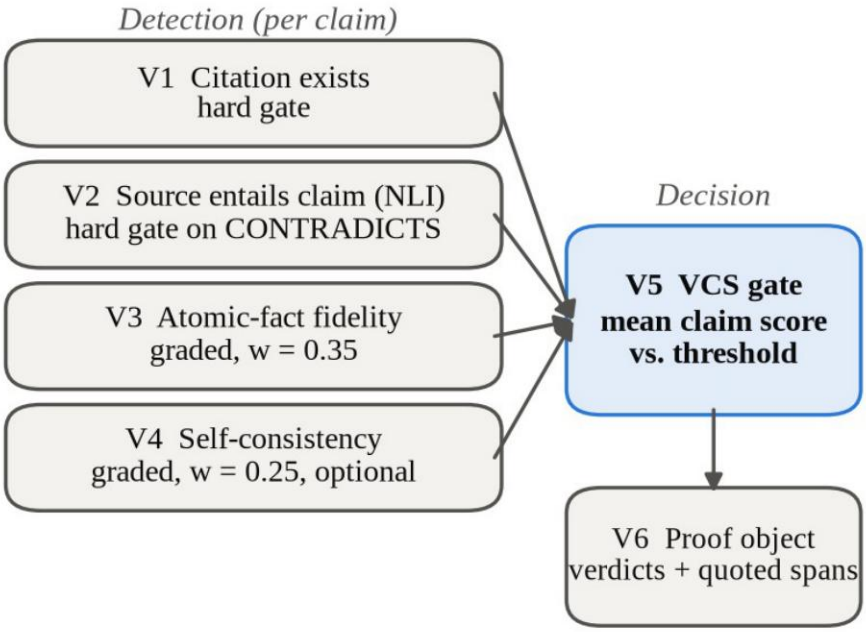


Fig. 4.2 The verification chain. V1–V4 check each claim, V5 turns the scores into one decision and V6 records the evidence.

4.7 ABSTENTION AND GENERAL-KNOWLEDGE ANSWERS

Refusing is the intended outcome when the documents do not support an answer. One kind of question is treated differently: general legal-concept questions such as “What is an indemnity clause?” are usually not defined in any uploaded contract. When such a question is declined by the gate, the system answers it from the model's own knowledge, labels the reply “General knowledge” and attaches no citations and no score, so it cannot be mistaken for a sourced answer. A question that refers to the documents (“this agreement”, “section 4”, “under the Act”) always stays on the verified path.

4.8 DATA FLOW AND UML DIAGRAMS

4.8.1 Context Diagram (DFD Level 0)

At the highest level (Figure 4.3), the user gives the system documents and questions and receives answers with a verdict and a proof. To produce them, the system exchanges prompts and answers with the external LLM provider and sends text to the local models, which return embeddings, relevance scores and entailment verdicts.

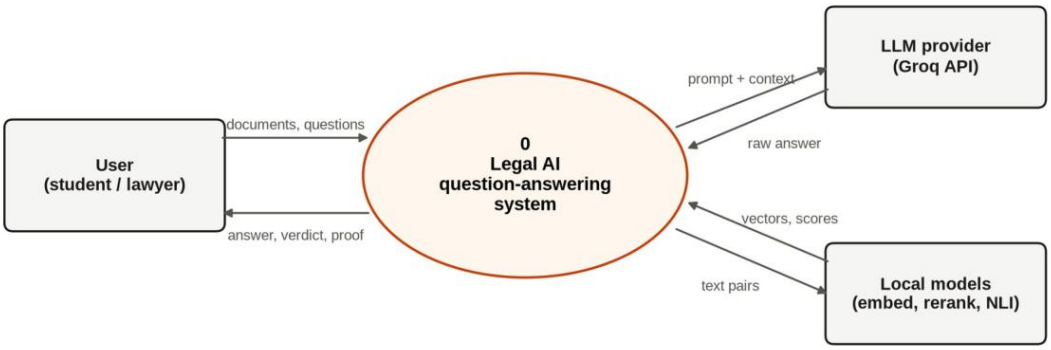


Fig. 4.3 Data flow diagram, level 0 (context diagram).

4.8.2 Level-1 Data Flow Diagram

Figure 4.4 breaks the system into six processes. Processes 1.0 and 2.0 run when a document is uploaded: the file is checked for duplicates, cleaned, chunked, embedded and written to the index (D1). Processes 3.0 to 6.0 run for every question: the question is matched against the index, the best chunks are given to the model, the model's claims are verified, and the final answer is returned with its proof and saved to the chat history (D2). Experiment results (D3) are written by the evaluation scripts and read by the Evaluation page.

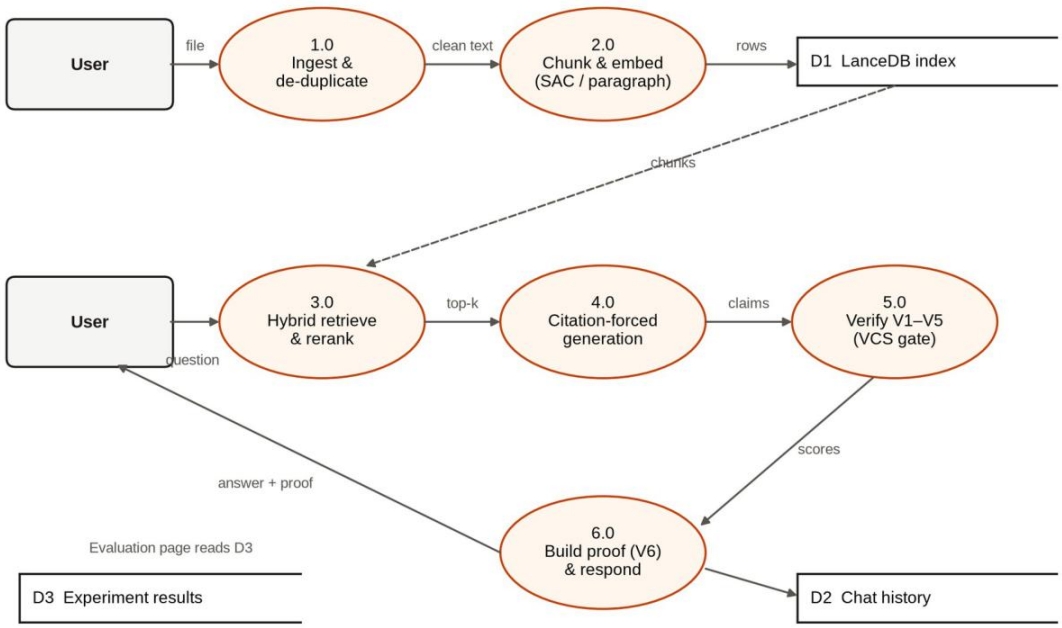


Fig. 4.4 Data flow diagram, level 1.

4.8.3 Use Case Diagram

The user is the main actor (Figure 4.5). The use cases correspond to the pages of the application: uploading, replacing and deleting documents, asking questions and viewing proofs, comparing retrieval methods, viewing evaluation results and managing saved chats. The LLM provider takes part in answering questions, and the local NLI model in answering and in building the proof.

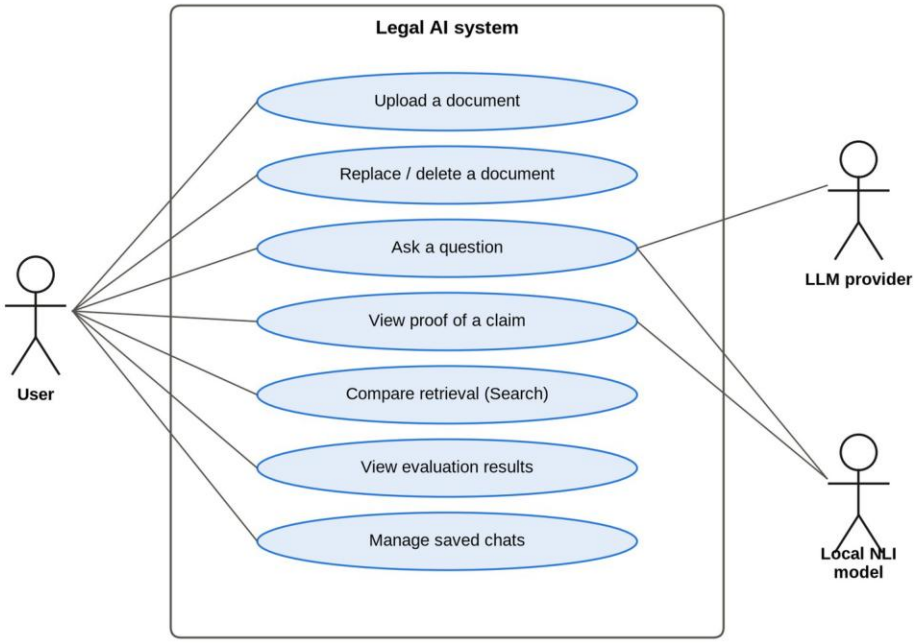


Fig. 4.5 Use case diagram.

4.8.4 Sequence Diagram for Asking a Question

Figure 4.6 shows the messages exchanged when a question is asked. The interface sends one request to the backend, which retrieves and reranks candidates, calls the LLM once, verifies each claim with the NLI model, applies the gate and returns the answer, its verdict and its proof in a single response. Keeping the whole exchange inside one request means the user never sees an unverified intermediate state.

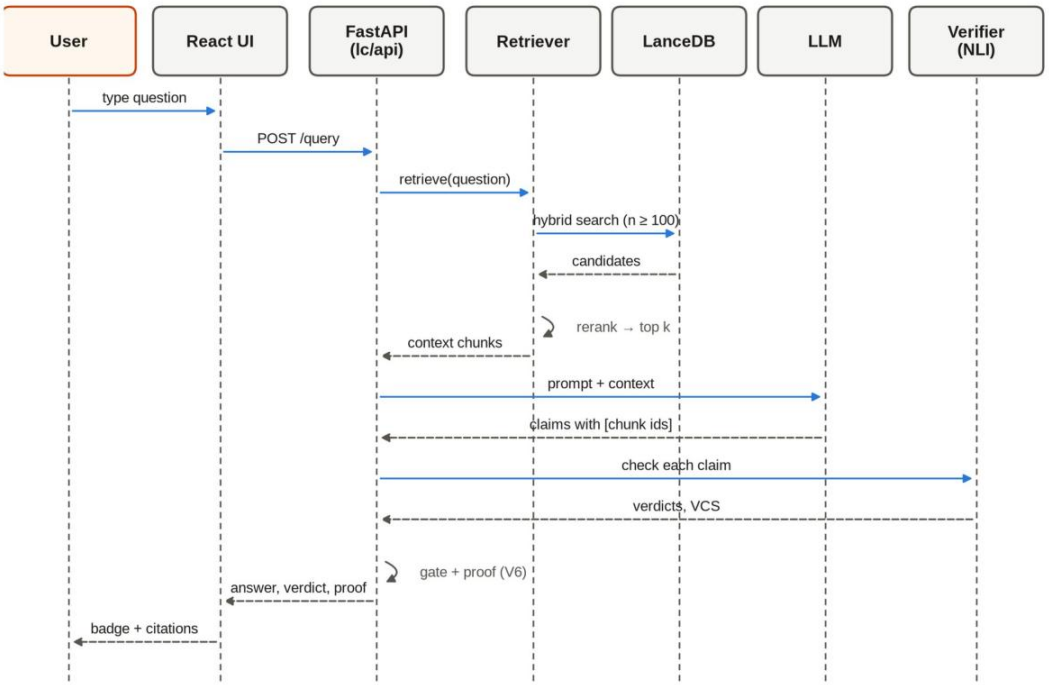


Fig. 4.6 Sequence diagram for answering a question.

4.9 MODULE DESIGN AND ORGANISATION

The code is organised into modules that follow the stages of the pipeline. Each has one responsibility and communicates through plain data (chunk records, answers, verification results), which is what made it possible to build the LangChain version stage by stage and to test every module on its own. Table 4.2 lists the modules with their inputs and outputs.

Table 4.2 Modules of the system

Module	**Responsibility**	**Input → Output**
Ingestion (ingestion/)	Load PDF/TXT, clean text, parse statute structure, SHA-256 duplicate check	File → cleaned text + document tree
Chunking (chunking/, lc/splitters.py)	Summary-augmented chunking for statutes; paragraph fallback	Document → chunk records with ids and summaries
Embedding and index (embedding/, indexing/, lc/vectorstore.py)	Embed summary + text; write, append and delete rows in LanceDB; rebuild the BM25 index	Chunks → index rows

Module	Responsibility	Input → Output
Retrieval (retrieval/, lc/retrieval.py)	Dense + BM25 search, RRF fusion, cross-encoder reranking, optional neighbour expansion	Question → top-k chunks
Generation (generation/, lc/generation.py)	Citation-forced prompt, LLM adapters, claim parser, general-knowledge path	Question + chunks → claims with citations
Verification (verification/, lc/verification.py)	V1–V6, VCS, gate, proof object	Claims + chunks → decision, score, proof
API (api/, lc/api.py)	FastAPI endpoints, dependency injection, error handling	HTTP request → JSON response
Chat store (chat/)	One JSON file per conversation; safe ids	Messages → saved conversations
Evaluation (evaluation/)	Retrieval metrics, ablation runs, experiment reader for the Evaluation page	Runs → results.json files and summaries
Web interface (web/)	Five React pages and shared components	User actions → API calls and views

4.10 BASELINES AND COMPARISON METHODS

Because the system is a pipeline rather than a trained model, its baselines are configurations of itself with one design decision removed, so that each decision can be credited or blamed on its own:

- **Chunking baseline:** paragraph chunking applied to every document, against summary-augmented chunking where structure exists. This isolates the contribution of SAC (Section 6.2).
- **Retrieval baselines:** dense-only against hybrid retrieval, and reranking on against off, in all four combinations (Section 6.2).
- **Verification baselines:** the full chain against the chain with each layer V1–V6 removed in turn, on the same answers (Section 6.3).
- **Framework baseline:** the plain-Python implementation against the LangChain implementation, on identical inputs, with five acceptance criteria fixed in advance (Section 6.6).

Comparison with published state-of-the-art systems is qualitative rather than numeric, because the published legal RAG systems [5, 6] target different languages, corpora and tasks, and no shared benchmark run was feasible within this project; Table 2.1 records how the design relates to each of them, and evaluation on the shared LegalBench-RAG benchmark [8] is listed as future work.

CHAPTER 5

IMPLEMENTATION

5.1 TECHNOLOGIES USED

Programming languages. Python 3.11 for the backend, the pipeline, the experiments and the tests; JavaScript (React) for the web interface.

Retrieval and machine learning. LanceDB as the embedded vector database with a BM25 full-text index in the same table; sentence-transformers for the all-MiniLM-L6-v2 embeddings and the BAAI/bge-reranker-base cross-encoder; Hugging Face Transformers (PyTorch) for roberta-large-mnli entailment; LangChain for loaders, the text-splitter interface, the vector store, retrievers, prompt templates, chat-model clients and LCEL composition; and the LLM providers Groq (openai/gpt-oss-120b), Google Gemini (gemini-flash-lite-latest in the ablation runs), OpenAI and Anthropic through adapters.

Backend and data. FastAPI and Uvicorn for the REST API; Pydantic for request and response models; pypdf for PDF text extraction; NumPy for the PCA projection of embeddings; JSON files for conversations and experiment results.

Frontend. React 18 with React Router, built with Vite; Tailwind CSS for styling and Framer Motion for small animations; a hand-drawn SVG scatter plot for the embedding map, without a charting library.

Testing and tooling. pytest for unit and API tests, with stand-in models so the tests need no downloads or keys; Playwright with Chromium to drive the real browser in the end-to-end test; Matplotlib for the result charts; Git and GitHub for version control.

5.2 PIPELINE CONFIGURATION

No model is trained in this project, so the usual training-setup table (optimizer, epochs, loss) does not apply. Its equivalent is the pipeline configuration: the fixed settings under which every experiment in Chapter 6 was run. Table 5.1 records them.

Table 5.1 Pipeline configuration used in the experiments

Parameter	Value
Embedding model	all-MiniLM-L6-v2 (384 dimensions, normalised)
Index	LanceDB embedded table: dense vectors + BM25 full-text index
Fusion	Reciprocal rank fusion, damping constant 60
Reranker	BAAI/bge-reranker-base cross-encoder
Context size k	12 chunks in the web interface; k = 5 in the retrieval ablation
Candidate pool	At least 100 (n = 50 in the retrieval ablation)
Chunk size (fallback)	Paragraphs packed into windows of at most 700 characters
NLI model	roberta-large-mnli
VCS weights	entailment 0.40, fidelity 0.35, consistency 0.25
Decision threshold τ	0.60 (uncalibrated default)
LLM (web interface, end-to-end test)	openai/gpt-oss-120b served by Groq
LLM (ablation runs)	gemini-flash-lite-latest (Google)

5.3 PROCEDURES

5.3.1 Project Planning

The work was divided into nine phases, each with its own goals and definition of done: (0) repository and environment set-up; (1) ingestion and SAC chunking; (2) embeddings and the LanceDB hybrid index; (3) fusion, reranking and retrieval-only evaluation; (4) citation-forced generation; (5) the V1–V6 verification chain, VCS and proof objects; (6) metrics and ablation experiments; (7) the FastAPI backend and React interface; and (8) consolidating the results into a paper. Later work added the LangChain version, the end-to-end test and a redesign of the interface. Every change was committed to Git with the tests passing, so the repository history records the build order.

5.3.2 Data Collection and Preparation

For controlled experiments we wrote five synthetic statutes in the style of Indian Acts, each with chapters, numbered sections and lettered or numbered clauses, and ten questions whose answers were labelled at clause level (fourteen gold clauses in all).

Writing the statutes ourselves meant the correct answer to every question was known exactly. Real documents were taken from ContractNLI, CUAD and MAUD. These are contracts without statute structure, so they exercise the paragraph fallback and show how the system behaves on documents it was not designed around.

5.3.3 Building the Pipeline

Each stage was implemented and tested before the next was started. Ingestion and chunking were tested on the synthetic statutes until every section and clause produced the expected chunk identifier. The index was built next, then retrieval, which was evaluated on its own before any model was called. Generation was added with a strict output format, and the verification chain was built layer by layer, with each layer switchable so that its effect could be measured later. Building in this order meant that when something failed, the failing stage was always the newest one, which kept debugging local.

5.3.4 Debugging Real-Document Failures

Testing on real contracts exposed defects that the synthetic statutes never triggered, and each of them showed up as a silent refusal rather than an error:

- The candidate pool feeding the reranker was too small, so a correct clause ranked low by the first stage could never be promoted. The pool was raised to at least 100 candidates.
- A reasoning model spent most of its output budget on internal reasoning and returned an answer cut off before any complete claim.
- A file name that began with a space carried the space into every chunk identifier, so V1 rejected correct citations. Identifiers are now trimmed.
- During the end-to-end test: the model once wrote citations in full-width brackets (【...】); a comma in a file name split a citation in two; and a question about the tenancy Act was routed to the general-knowledge path. All three were fixed, each with a test that failed before the fix and passes after it.

In each case the cause was found by inspecting the intermediate output (the retrieved list, the token counts, the identifiers) before changing any code. This discipline — measure the failing stage first, then edit — repeatedly prevented plausible-looking fixes to the wrong component.

5.4 KEY CODE SNIPPETS

Four short excerpts from the repository show how the central design decisions look in code. The first is the system prompt that enforces citation discipline; everything the verification chain does depends on the model obeying this format (or being caught when it does not):

```
ABSTENTION_MARKER = "INSUFFICIENT_CONTEXT"

SYSTEM_PROMPT = f"""You are a legal question-answering assistant.
You answer ONLY from the provided context passages, never from
prior knowledge.

Rules you must follow exactly:
1. Use ONLY the information in the CONTEXT passages below.
2. Write each factual claim on its own line.
3. End every claim line with a citation to the chunk id(s) it is
   supported by, in square brackets, e.g.
   [urban_tenancy_act_2019::s4:b]. If a claim draws on more than
   one passage, cite them all: [id_one][id_two].
4. Never write a factual sentence without a citation.
5. If the context does not contain enough information to answer,
   respond with a single line beginning exactly with
   "{ABSTENTION_MARKER}:" followed by a short reason."""
```

The second is the configuration of the verification chain. The weights and threshold of equations (4.2) and (4.3) live in one dataclass, and each layer has a switch, which is what the ablation study drives:

```
@dataclass
class VCSConfig:
    w_entail: float = 0.40
    w_fidelity: float = 0.35
    w_consistency: float = 0.25
    threshold: float = 0.60

    # Per-layer switches, used by the phase-6 ablation sweep.
    enable_v1: bool = True   # citation-existence hard gate
    enable_v2: bool = True   # entailment verdict + CONTRADICTS
gate
    enable_v3: bool = True   # atomic-fact fidelity term
    enable_v4: bool = True   # self-consistency resamples
    enable_v5: bool = True   # threshold gating
    enable_v6: bool = True   # Proof Object emission
```

The third is hybrid search. Because LanceDB keeps the vectors and the BM25 index in one table, the fused query is a few lines; the RRF fusion of equation (4.1) is applied by the database's default reranker:

```
def search_hybrid(table, query_text, k=5, embedder=None):
    """Hybrid search: vector + FTS, fused with RRF."""
```

```
    embedder = embedder or SentenceTransformerEmbedder()
    vector = embedder.embed([query_text])[0]
    results = (
        table.search(query_type="hybrid")
        .vector(vector)
        .text(query_text)
        .limit(k)
        .to_list()
    )
    return [_row_to_dict(r) for r in results]
```

The fourth is the heart of summary-augmented chunking: each section emits one chunk per body and clause, and every chunk carries the extractive contextual summary that is embedded together with its text:

```
def chunk_section(document, chapter, section):
    """Emit chunk records for one section: a body chunk (if
    any) + one per clause."""
    chunks = []
    summary = contextual_summary_for_section(section)

    if section.body_text:
        chunks.append(ChunkRecord(
            chunk_id=_chunk_id(document.source_id, section),
            text=section.body_text,
            contextual_summary=summary,
            metadata=build_metadata(document, chapter, section),
        ))
```

The full source code is in the GitHub repository (Appendix A).

5.5 WEB APPLICATION AND DEPLOYMENT

The FastAPI backend exposes each stage as an endpoint, and the React interface was built around the stages rather than as a plain chat box: the proof of every claim is folded into the answer, and the Search page shows retrieval on its own. The front-end is a single-page React application built with Vite and styled with Tailwind CSS in a white-and-orange theme. It has five pages:

- **Home:** shows how many documents and chunks are indexed and summarises the three stages of the system: retrieve, cite and verify (Figure 5.1).
- **Ingest:** uploads a PDF or text file, reports how many chunks were created and whether statute structure was found, and offers to replace a document that is already indexed (Figure 5.2).

- **Ask:** a chat page with saved conversations. Each answer carries a badge (Verified, Abstained or General knowledge), its VCS and clickable citations that open the proof.
- **Search:** runs one query through dense, keyword and hybrid retrieval side by side, so retrieval can be inspected on its own.
- **Evaluation:** a 2D map of every indexed chunk and one card for every logged experiment, with its description and headline numbers.

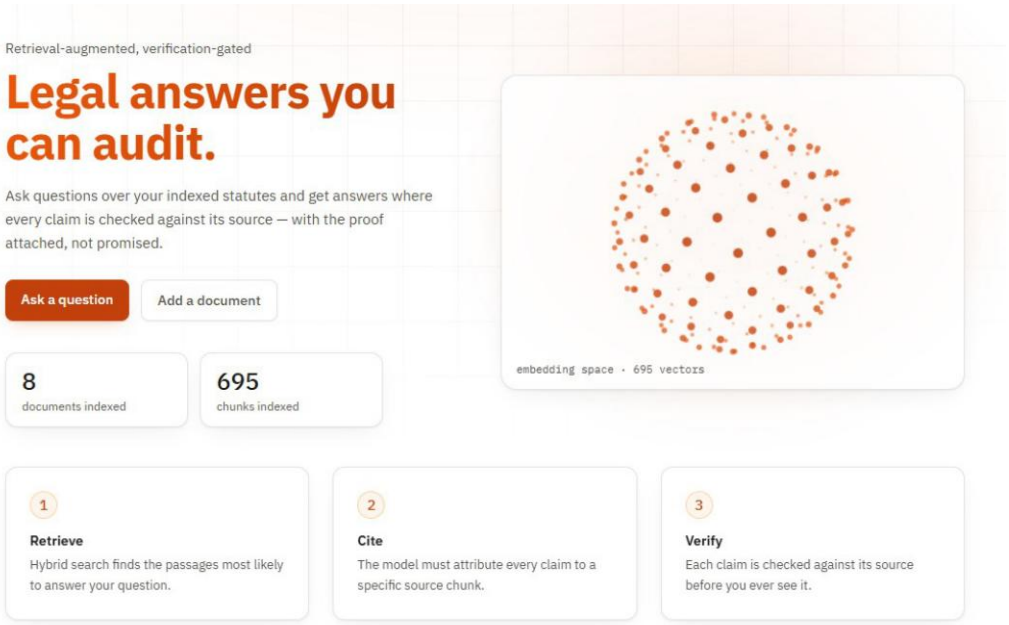


Fig. 5.1 Home page: index size and the three stages of the system.

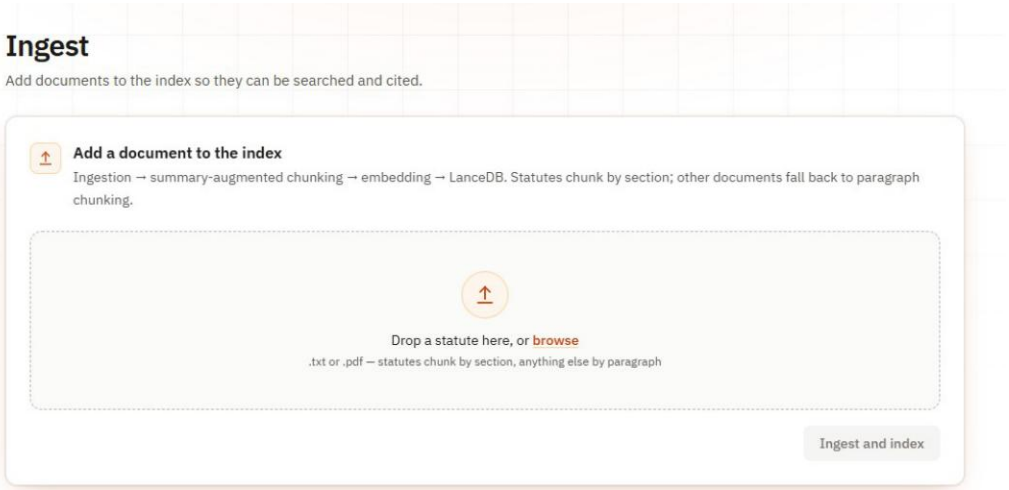


Fig. 5.2 Ingest page: drag-and-drop upload of a PDF or text file.

The backend exposes endpoints to ingest and delete documents, to retrieve passages, to answer a question through the full pipeline, to read index statistics and the embedding map, to list logged experiments and to store conversations. Models are created lazily and injected as dependencies, so the automated tests replace them with small stand-ins and run without downloads or API keys. Each failure is returned with its real reason: for example, a duplicate upload returns HTTP 409, a missing API key 503 and an empty index 409. The full list of endpoints is given in Appendix A.5. For deployment on a development machine, a start.bat script starts the backend and interface together on Windows; the first start downloads the local models once.

5.6 THE LANGCHAIN VERSION

The pipeline exists in two versions. The default one (lc/) composes loading, retrieval and generation with LangChain [19]: document loaders, custom text splitters, HuggingFace embeddings, the LanceDB vector store, an ensemble retriever with a cross-encoder reranker, and an LCEL chain for generation. The section-aware chunking and the verification chain have no LangChain equivalent and remain our own Python code, which the LangChain version calls directly. The plain-Python version (src/) is kept as a reference.

The LangChain version was built in a separate folder and branch so that the plain-Python version stayed untouched. Five acceptance criteria were written down before any measurement: equal retrieval quality, equal safety, all tests passing with the interface working end to end, no more than 10% slower, and every intermediate step still visible. The LangChain version met all five (Section 6.6) and became the default.

5.7 TESTING AND VALIDATION

5.7.1 Design of Test Cases and Scenarios

Testing was done at three levels: automated unit and API tests, experiments that measure each part of the system, and an end-to-end test through the browser. Table 5.2 summarises the automated tests. After the fixes made during the end-to-end test, the plain-Python suite passed 263 tests (1 skipped) and the LangChain suite passed 113.

Table 5.2 Automated test suites

Area	What is checked
Ingestion and chunking	Statute parsing, SAC chunk ids and summaries, paragraph fallback, file names with spaces or commas
Index and retrieval	Dense, BM25 and hybrid search, reranking, neighbour expansion, duplicate detection, deletion
Generation	Prompt format, claim parsing (including full-width brackets and commas in ids), abstention marker, general-knowledge routing, provider factory
Verification	Each of V1–V6, hard gates, VCS weighting when layers are skipped, the threshold decision, proof contents
API	Every endpoint, error codes (400, 409, 415, 422, 503), chat storage, evaluation reader, embedding map
LangChain parity	Identical chunks and parsed claims between the LangChain and plain-Python versions

The end-to-end test used fifteen questions written before any document was uploaded, covering six categories with a stated expected behaviour for each (Table 5.3).

Table 5.3 End-to-end test scenarios

Category	Count	Expected behaviour
Specific fact	5	Correct answer, shown as Verified, citing the right passage
Comparison across documents	2	Correct answer combining several agreements
Unanswerable	3	Abstain: the documents do not contain the answer
General concept	2	Answer labelled General knowledge, without citations
False premise	2	Abstain or correct the premise; never confirm it as Verified
Paraphrase	1	Correct answer although none of the source's keywords are used

The same run also tested the rest of the application: uploading a document twice (a duplicate warning is expected), replacing it (the same chunk count is expected), the Home counters, the Search page, the Evaluation page, chat history after a reload, and deleting a document (a question about it should then abstain).

5.7.2 Validation

- **Answers fixed in advance:** the expected answers and source passages for the end-to-end test were committed to the repository before anything was uploaded, so they could not be adjusted afterwards.
- **Ablations:** each retrieval setting and each verification layer was switched off in turn, with everything else unchanged, to measure its contribution.
- **Corrupted claims:** 28 claims that must never be shown were created deliberately, to test whether the chain catches them.
- **Two implementations:** the LangChain and plain-Python versions were compared on the same questions and the same cached answers.
- **Logged results:** every experiment writes its settings and results to the experiments/ folder, and every number in this report is taken from those files.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 OUTPUT OF THE WORKING SYSTEM

This section shows the working application; all screenshots were taken from the running system. The Home and Ingest pages were shown in Chapter 5 (Figures 5.1 and 5.2). Figure 6.1 shows the duplicate guard: uploading the same document twice is refused, and the interface offers to replace the existing copy instead, so the index never holds a document twice.

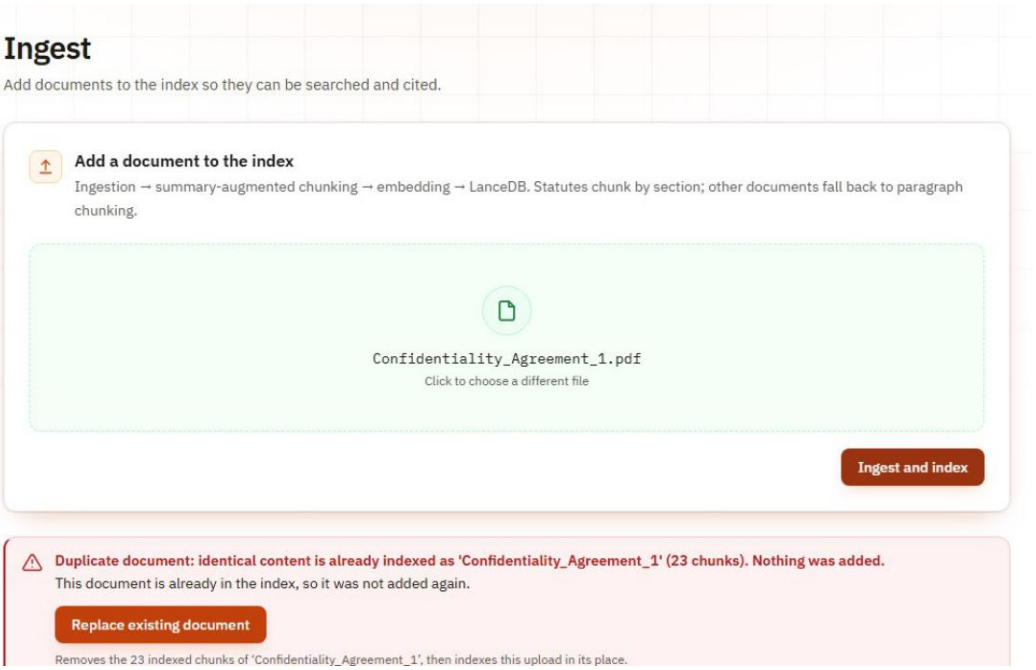


Fig. 6.1 Uploading the same document twice: the duplicate is refused and can be replaced.

Figure 6.2 shows the system's central artefact: a verified answer on the Ask page with the proof of the first claim open. The badge gives the verdict and the VCS, each claim ends with its citation, and the opened proof shows the quoted passage together with the verdict of every check — citation existence, entailment and fidelity — so the reader can audit the claim rather than trust it.

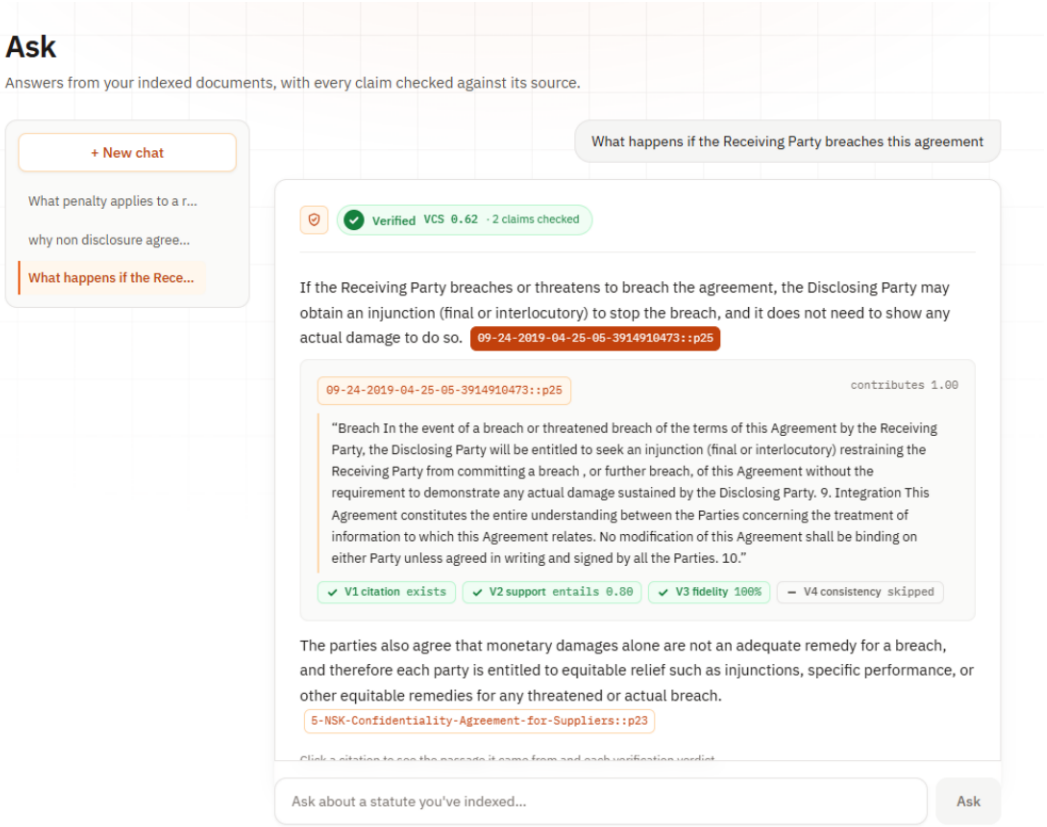


Fig. 6.2 Ask page: a verified answer with the proof of the first claim open, showing the quoted passage and the verdict of each check.

Figure 6.3 shows the other intended outcome: an abstention. The indexed documents do not cover the question, so the system asserts nothing and says so explicitly. Figure 6.4 shows that abstention is not mere keyword matching: a question paraphrased in everyday words (“thrown out by force”) is answered from the eviction clause at VCS 1.00, because dense retrieval bridges the vocabulary gap and the verification chain confirms the claim.

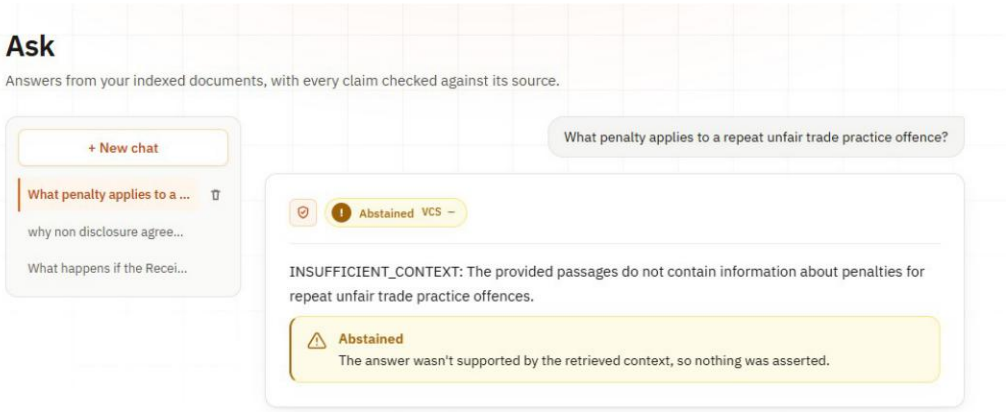


Fig. 6.3 An abstention: the indexed documents do not cover the question, so nothing is asserted.

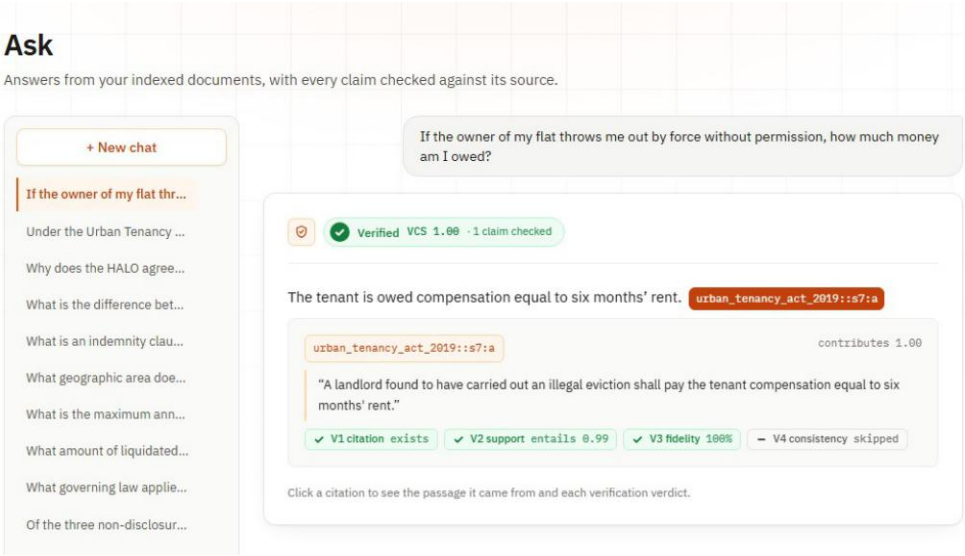


Fig. 6.4 A paraphrased question ("thrown out by force") answered from the eviction clause at VCS 1.00.

Figure 6.5 shows the Search page, which runs the same query through dense, keyword and hybrid retrieval side by side so that retrieval can be inspected on its own, and Figure 6.6 shows the Evaluation page: a 2D embedding map of every indexed chunk, and one card per logged experiment with its description and headline numbers.

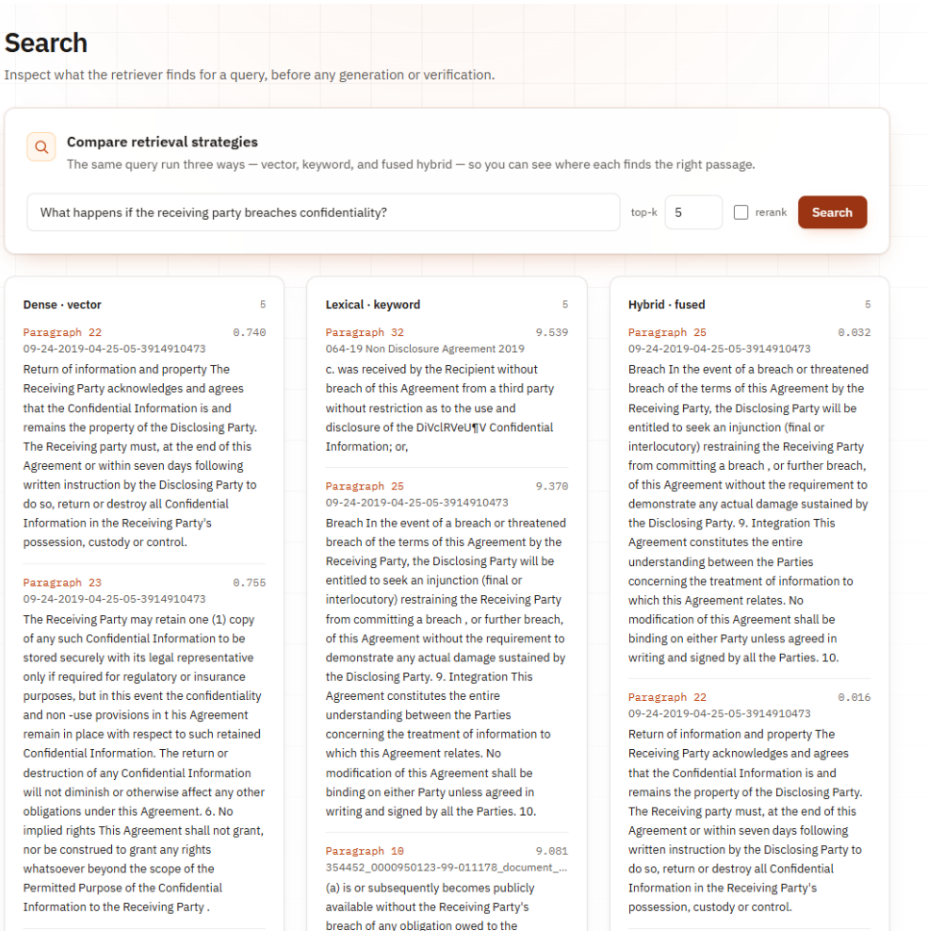


Fig. 6.5 Search page: the same question through dense, keyword and hybrid retrieval.

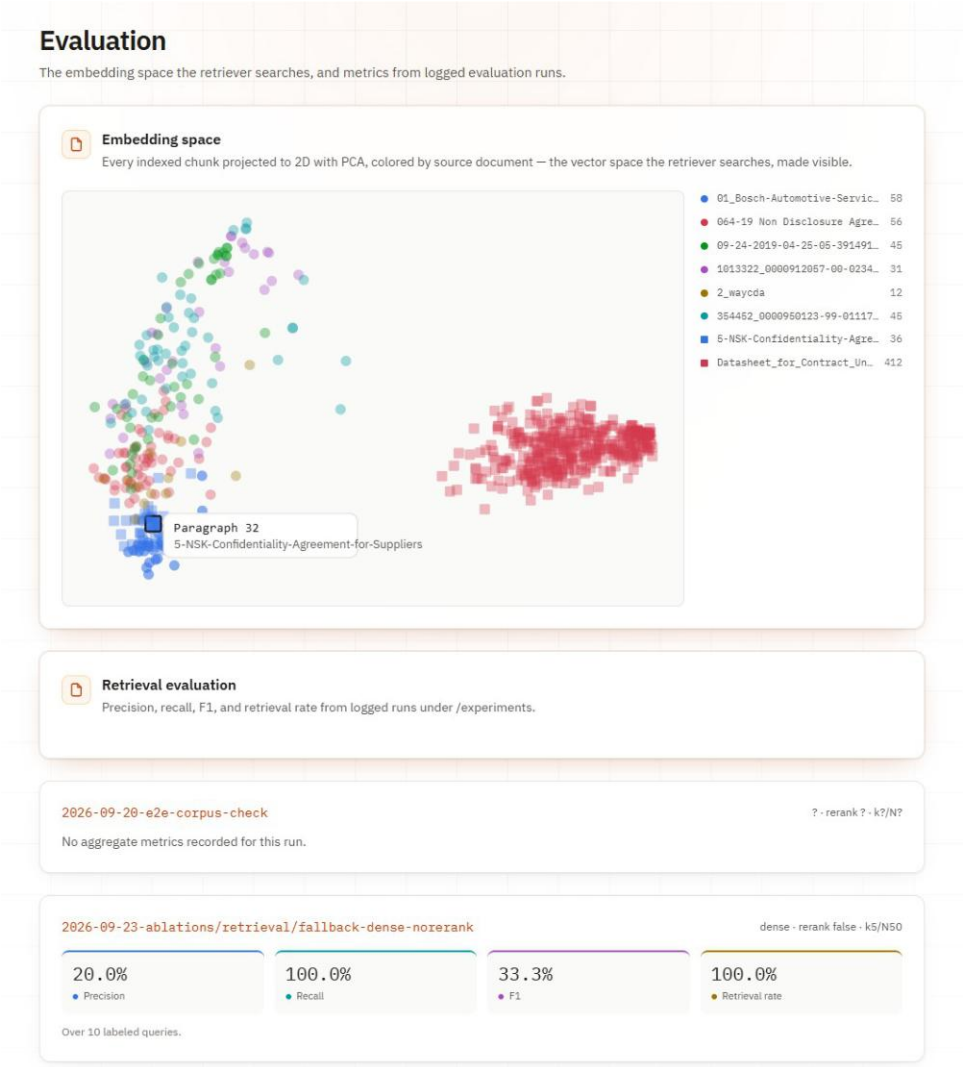


Fig. 6.6 Evaluation page: the embedding map of all chunks and the cards of the logged experiments.

6.2 RETRIEVAL AND CHUNKING

Table 6.1 compares the two chunking methods under four retrieval settings on the ten labelled questions (k = 5 chunks, candidate pool n = 50). Retrieval rate was 100% in every configuration.

Table 6.1 Retrieval results over 10 questions (k = 5, n = 50)

Chunking	**Retrieval**	**Rerank**	**Precision**	**Recall**	**F1**
SAC	Dense	Yes	0.280	1.000	0.417
SAC	Dense	No	0.280	1.000	0.417
SAC	Hybrid	Yes	0.280	1.000	0.417

Chunking	Retrieval	Rerank	Precision	Recall	F1
SAC	Hybrid	No	0.260	0.967	0.392
Paragraph	Dense	Yes	0.200	1.000	0.333
Paragraph	Dense	No	0.200	1.000	0.333
Paragraph	Hybrid	Yes	0.200	1.000	0.333
Paragraph	Hybrid	No	0.200	1.000	0.333

SAC reached an F1 of 0.417 in three of the four settings against 0.333 for paragraph chunking in all four (Figure 6.7). Recall was close to 1.000 everywhere, so the difference comes from precision: at a fixed k, smaller and more focused chunks give higher precision. SAC also passed about 550 characters of context per question to the model against 1,500 to 1,800 for paragraph chunks — a third of the text with no loss of recall, which matters both for cost and for keeping the model's attention on relevant text.

The question set turned out to be too easy to separate the retrieval settings themselves, since every setting found a gold clause for every question, so no conclusion about dense versus hybrid retrieval is drawn from it. Reranking had a clear cost: 6.7 to 12.8 seconds per question on a laptop CPU, against less than 0.1 seconds without it.

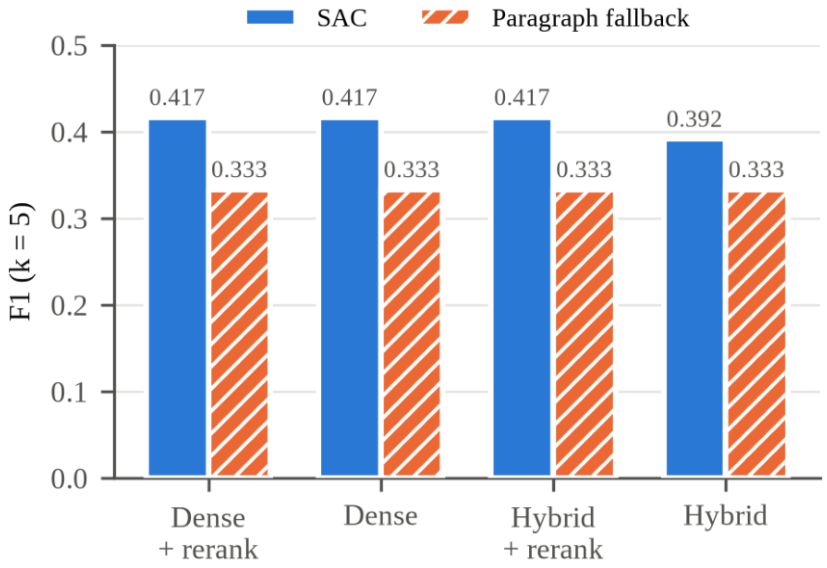


Fig. 6.7 Retrieval F1 for SAC and paragraph chunking under each retrieval setting.

6.3 VERIFICATION ABLATION

To see what each layer catches, the cached answers to the ten questions were corrupted in two ways: every citation was replaced with an identifier that was never retrieved (14 claims), and every claim was swapped with a claim from a different question while keeping its citation (14 claims). Table 6.2 removes one layer at a time.

Table 6.2 Verification ablation (clean set: 10 questions, 14 claims; bad claims: 28)

Configuration	**Mean VCS**	**Answered**	**Refused**	**Bad claims shown**
Full chain	0.768	7	3	0 / 28
Without V1	0.768	7	3	0 / 28
Without V2	0.813	8	2	0 / 28
Without V3	0.777	7	3	0 / 28
Without V4	0.723	7	3	0 / 28
Without V5 (gate)	0.768	10	0	28 / 28
Without V6	0.768	7	3	0 / 28

The full chain showed none of the 28 corrupted claims, and neither did any configuration with a single detector removed, because the detectors overlap: without V1, a fabricated citation still scores zero because V2 finds no premise and V3's fidelity collapses. Removing the gate (V5) let all 28 through (Figure 6.8). The detectors still scored them at zero, but a score changes nothing unless something acts on it. **The gate, not any single detector, is what protects the user.** On the clean answers, V2 changed decisions the most: without it, one more question was answered and the mean VCS rose from 0.768 to 0.813.

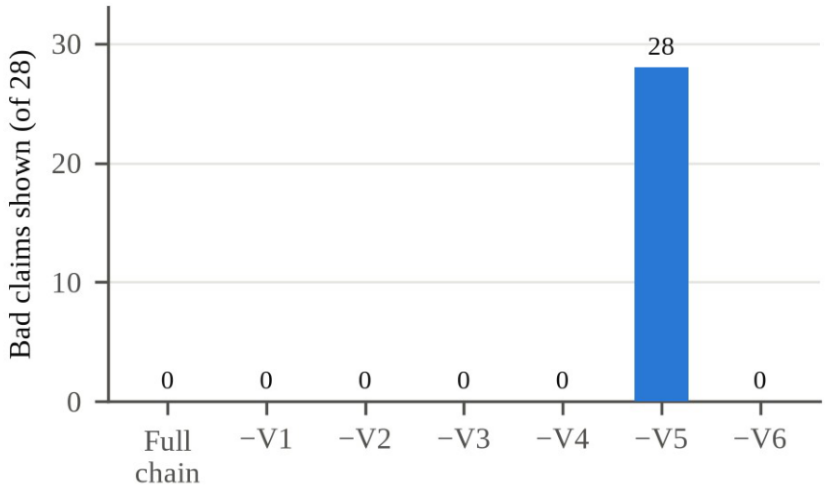


Fig. 6.8 Corrupted claims shown to the user when each layer is removed (28 in total).

6.4 ERROR ANALYSIS: FALSE ABSTENTIONS

All ten questions retrieved their gold clause, yet the full chain answered only seven. In each of the three refusals the NLI model judged a correctly cited claim NEUTRAL instead of ENTAILS; one question on the grounds for eviction scored zero on all three of its claims without any hard gate firing. The threshold of 0.60 is a default that has not been calibrated, and these refusals are the clearest evidence that it needs to be.

A further weakness of averaging appears in Figure 6.2: that answer's VCS of 0.62 is the mean of two claims, the first scoring 1.00 and the second therefore below 0.25. The weak claim was shown only because a strong claim sat next to it; gating each claim on its own score would have withheld it. These two failure directions — refusing answerable questions, and letting a weak claim ride on a strong one — are both properties of the scoring rule rather than of any detector, which is why re-designing the aggregation (per-claim gating, or minimum instead of mean) heads the future-work list.

6.5 BATCH TEST ON REAL CONTRACTS

Ten ContractNLI agreements were indexed and five specific and five general questions were asked. Rate limits on the free API tier interrupted some runs, so the results were collected over three runs. Three of the five specific questions were verified; citations were logged for one of them (the parties to the Federal Express agreement, verified at VCS 1.00 with the correct citation). Two were refused although the answer was in the index, one at a VCS of 0.47. Four of the five general questions were answered from model knowledge and labelled as such; the fifth, “What is confidential information?”, was answered from the documents at VCS 0.99 because the agreements define the term. No general answer was presented as sourced.

6.6 END-TO-END TEST THROUGH THE INTERFACE

Five documents were indexed through the Ingest page (three ContractNLI non-disclosure agreements, one CUAD non-competition agreement and the synthetic tenancy Act) and the fifteen questions of Table 5.3 were asked through the Ask page, each in a new chat, on the LangChain backend with Groq's openai/gpt-oss-120b model. Each answer was compared with its expected answer and source passage (Table 6.3).

Table 6.3 End-to-end test results (15 questions; “after” re-asks the three questions affected by the fixes)

Outcome	**Before fixes**	**After fixes**
Correct, shown as Verified	3	4
Wrong, shown as Verified	0	0
Correct abstention	4	5
False abstention	4	3
General knowledge, correctly labelled	2	2
Other	2	1

No wrong or mis-cited answer was ever shown as verified, and every verified claim was supported by the passage its proof quoted. Two of the three unanswerable questions and both false-premise questions were refused, the general-concept questions were answered and labelled, and the paraphrased question was answered correctly at VCS 1.00 (Figure 6.4). The duplicate upload was refused, replacing it restored the same 23 chunks, and deleting a document removed its chunks, after which a question about it abstained. The mean response time was 17.3 seconds per question (median 16.1 s), almost all of it spent in the model call.

The main failure was false abstention, with four distinct causes. Two were parser bugs, fixed as described in Section 5.3.4; passing the recorded answers through the fixed code gave a VCS of 1.00 for both. A third was the NLI model scoring a correct claim at 0.51, or judging a claim NEUTRAL because the cited clause does not repeat the party's name, which appears only in the agreement's heading. The fourth was comparison questions: one candidate pool serves the whole question and can be filled by some of the documents, so another never reaches the model.

The same limit produced the most instructive result (Figure 6.9): asked for the governing law of three agreements, the system showed two correct, verified answers, while the model's note that the third was missing had no citation and was dropped. Nothing false was shown, but verification checks what an answer says, not what it leaves out. Completeness is a different property from faithfulness, and the current chain guarantees only the latter.

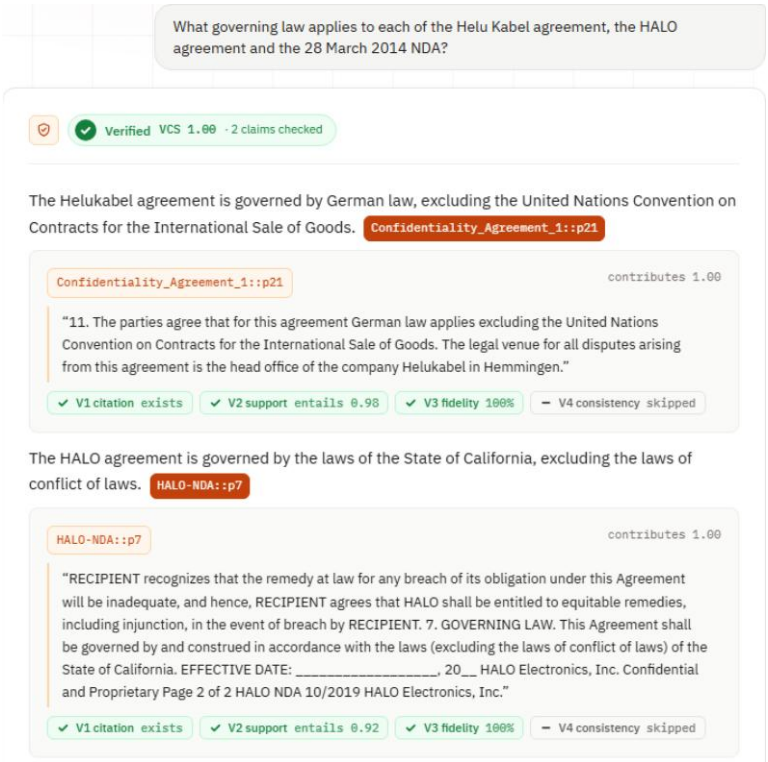


Fig. 6.9 A verified but incomplete answer: both claims are correct, but one of the three agreements asked about is missing.

6.7 LANGCHAIN VERSION COMPARED WITH PLAIN PYTHON

Table 6.4 reports the five acceptance checks that were fixed before any measurement (Section 5.6). The LangChain version reproduced the plain-Python results exactly where exactness was required, and stayed within the allowed margin on speed.

Table 6.4 Acceptance checks for the LangChain version

Check	**Plain Python**	**LangChain**	**Result**
Retrieval F1 / recall (10 questions)	0.417 / 1.000	0.417 / 1.000	Pass: same top-5 in the same order for all 10
Corrupted claims shown	0 / 28	0 / 28	Pass
Clean mean VCS, answered / refused	0.768, 7 / 3	0.768, 7 / 3	Pass: identical decisions for all 10
Tests and web interface	—	All tests pass; 5 of 5 interface steps	Pass
Mean query time (20 requests each)	13.86 s	14.50 s	Pass: +4.6%, within the variation of the LLM calls

Check	Plain Python	LangChain	Result
Intermediate steps visible	—	Candidates, prompt, raw output, verdicts	Pass

The comparison found one real defect in the LangChain version: the Groq client rejected an unset temperature, which the stubbed tests had not exercised. It was fixed and a test now builds every provider's client. The LangChain version was adopted as the default; the plain-Python version remains as a reference.

6.8 ETHICAL AND RESPONSIBLE AI CONSIDERATIONS

Not legal advice. The system is a research prototype and must not be used as a substitute for legal advice. Its value is that it makes its evidence visible and refuses when that evidence is weak, which leaves the final judgement with a person who can read the source. A verified answer means the cited passage supports the claim, not that the legal conclusion is complete.

Data privacy. Documents uploaded to the system may be confidential. The index is stored locally and never leaves the machine, but questions and retrieved passages are sent to the chosen LLM provider, which users should be aware of; a local LLM option is listed as future work precisely so that confidential documents need never leave the user's machine. No personal data is collected by the system itself, and the datasets used are public research corpora used under their licences.

Bias and coverage. The system answers only from the documents it is given, which bounds — but does not eliminate — the risk of biased output: the choice of documents determines the answers, and a one-sided document set yields one-sided answers with honest citations. The general-domain NLI model is another source of uneven behaviour: Chapter 6 shows it is conservative on legal phrasing, which produces false refusals rather than false answers — the safer direction, but still a bias to document.

Misuse and honest limits. The clearest misuse risk is over-reliance: treating a “Verified” badge as legal certainty. The interface mitigates this by exposing the proof of every claim and by labelling general-knowledge answers as unverified. The evaluation in this chapter is deliberately reported with its failures — false abstentions, an incomplete comparison answer, rate-limit interruptions — so that the system's limits are part of its record, not a surprise for its users.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

This project set out to build a legal question-answering system that does not simply trust its language model. Legal AI retrieves passages from statutes and contracts with structure-aware chunking and hybrid search, requires the model to cite a passage for every claim, and checks each claim through a six-layer verification chain before deciding whether to show the answer. Every answer carries a verdict, a score and a proof that lets the reader see the exact evidence.

The experiments support the design. Summary-augmented chunking improved retrieval F1 from 0.333 to 0.417 while sending the model a third of the text. The verification ablation showed that no single detector is essential, because they overlap, but that the gate is: with the gate the system showed none of 28 corrupted claims, and without it, all 28. In the end-to-end test on real documents with answers fixed in advance, no wrong answer was shown as verified. The LangChain version reproduced the plain-Python results exactly, so the framework could be adopted without changing the system's behaviour.

In summary, the project shows that a small, locally run pipeline can make legal answers checkable: the combination of citation-forced generation, overlapping detectors and a single gate kept every corrupted and every wrong answer away from the user in our tests. The most important next steps are to reduce false refusals and to evaluate the system on larger, real legal collections.

7.2 LIMITATIONS

The results also show clear limits, and honesty about them is part of the project's method:

- **Cautious to a fault:** the system refused three of ten answerable questions in the ablation and four of fifteen in the end-to-end test, mainly because a general-domain NLI model judged correct claims as neutral.

- **Uncalibrated threshold:** the 0.60 gate is a default; no held-out calibration set has been used to tune it.
- **Averaging can hide a weak claim:** a weak claim can be shown beside a strong one, because the gate acts on the mean rather than on each claim.
- **Completeness is not checked:** a partial answer to a comparison question can look complete; verification checks what an answer says, not what it leaves out.
- **Small evaluation:** ten labelled questions, five synthetic statutes and fifteen end-to-end questions judged by one reviewer — the numbers are observations, not rates.

7.3 FUTURE ENHANCEMENTS

The limitations found in the experiments point to concrete extensions.

Verification

- Calibrate the threshold on a held-out set that includes unanswerable questions, so that correct refusals are counted as well as false ones.
- Gate each claim separately, or combine claim scores with the minimum instead of the mean, so one strong claim cannot carry a weak one.
- Use a legal-domain NLI model, or fine-tune one on legal text, to reduce NEUTRAL verdicts on correct claims.
- Add the document title to the NLI premise, as SAC already does for retrieval, so a claim that names a party can be checked against a clause that does not repeat the name.
- Flag partial answers: show a warning when the model reports that part of the question could not be answered.

Retrieval and Data

- Per-document retrieval for comparison questions, so that every agreement asked about reaches the model.
- Evaluate on LegalBench-RAG and on Indian statutes from India Code, the corpus the project was first designed for, with a larger and harder question set.

- Test neighbour-chunk expansion, which is implemented but switched off, on questions whose answers span adjacent chunks.
- Support more formats, starting with HTML, so that the full ContractNLI release can be indexed.

Product and Deployment

- Deployment on a server with the models preloaded, with authentication and per-user document collections.
- Local LLM option, so that confidential documents never leave the user's machine.
- Replace the langchain-community components, which are being wound down by their maintainers, with standalone integration packages.
- Multilingual support, beginning with Indian languages, for statutes published in regional languages.

REFERENCES

[1] Mata v. Avianca, Inc., No. 22-cv-1461 (PKC) (S.D.N.Y. 2023).

[2] V. Magesh, F. Suzgun, M. Dahl, M. Zheng, C. D. Manning, and D. E. Ho, "Hallucination-free? Assessing the reliability of leading AI legal research tools," *Journal of Empirical Legal Studies*, 2025.

[3] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[4] M. Reuter, T. Lingenberg, R. Liepina, F. Lagioia, M. Lippi, G. Sartor, A. Passerini, and B. Sayin, "Towards reliable retrieval in RAG systems for large legal datasets," in *Proc. Natural Legal Language Processing Workshop (NLLP)*, 2025, pp. 17–30.

[5] A. Figueiredo, J. Lucas, T. Ribeiro et al., "Grounded in law: A multi-stage anti-hallucination pipeline for legal RAG systems in Brazilian Portuguese," in *Proc. PROPOR*, 2026.

[6] J. Maghakian, R. Sinha, M. Schettewi, and G. Kaur, "Embedding-free RAG," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 24974–24985.

[7] C. Niu et al., "RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models," in *Proc. ACL*, 2024.

[8] N. Pipitone and G. H. Alami, "LegalBench-RAG: A benchmark for retrieval-augmented generation in the legal domain," arXiv, 2024.

[9] Y. Koreeda and C. D. Manning, "ContractNLI: A dataset for document-level natural language inference for contracts," in *Findings of EMNLP*, 2021.

[10] D. Hendrycks, C. Burns, A. Chen, and S. Ball, "CUAD: An expert-annotated NLP dataset for legal contract review," in *NeurIPS Datasets and Benchmarks Track*, 2021.

[11] S. H. Wang et al., "MAUD: An expert-annotated legal NLP dataset for merger agreement understanding," in *Proc. EMNLP*, 2023.

[12] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019.

[13] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proc. ACM SIGIR*, 2009.

[14] S. Xiao et al., “C-Pack: Packaged resources to advance general Chinese embedding,” arXiv:2309.07597, 2023.

[15] Y. Liu et al., “RoBERTa: A robustly optimized BERT pretraining approach,” arXiv:1907.11692, 2019.

[16] A. Williams, N. Nangia, and S. R. Bowman, “A broad-coverage challenge corpus for sentence understanding through inference,” in *Proc. NAACL-HLT*, 2018.

[17] S. Min et al., “FActScore: Fine-grained atomic evaluation of factual precision in long form text generation,” in *Proc. EMNLP*, 2023.

[18] P. Manakul, A. Liusie, and M. J. F. Gales, “SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models,” in *Proc. EMNLP*, 2023.

[19] H. Chase, “LangChain,” 2022. [Online]. Available: https://github.com/langchain-ai/langchain (accessed 2026).

APPENDICES

APPENDIX A – PROJECT FILES, SETUP AND EXECUTION GUIDE

A.1 GitHub Repository

The complete source code, the synthetic statutes, the labelled question sets, every logged experiment and the documentation are in the public GitHub repository:

```
https://github.com/vvvvvivekkk/LEGAL-AI-PROG
```

The repository README lists the setup steps, the dataset locations and the commands to reproduce the results. Table A.1 shows the layout.

Table A.1 Main folders and files of the repository

Path	**Contents**
lc/	LangChain version of the pipeline and its FastAPI app (default backend)
src/	Plain-Python reference: ingestion, chunking, embedding, indexing, retrieval, generation, verification, API, chat store, evaluation
web/	React interface (pages, components, API client); web/.env sets the backend address
tests/, lc/tests/	Automated tests for both versions
data/sample/	The five synthetic statutes
data/eval/	Labelled question set and the cached answers used by the ablation
experiments/	Every logged experiment (settings, results, screenshots) and a README explaining each run
docs/	Architecture notes, phase documents, system guide and paper
start.bat	Starts the backend and the web interface together on Windows
requirements.txt, lc/requirements.txt	Python dependencies of the two versions
.env.example	Template for the LLM provider and API keys

A.2 Required Packages

Plain-Python backend (requirements.txt): lancedb, sentence-transformers, rank_bm25, fastapi, uvicorn, python-multipart, httpx, pytest, pypdf, python-dotenv, google-genai, openai. The LangChain version adds (lc/requirements.txt): langchain, langchain-core, langchain-community, langchain-text-splitters, langchain-huggingface, langchain-groq, langchain-google-genai, langchain-openai and langchain-anthropic. The web interface needs react, react-dom, react-router-dom and framer-motion, with vite and tailwindcss for building.

A.3 Environment Setup

```
git clone https://github.com/vvvvvivekkk/LEGAL-AI-PROG.git
cd LEGAL-AI-PROG

python -m venv .venv-lc
.venv-lc\Scripts\pip install -r lc\requirements.txt

copy .env.example .env   (then set LLM_PROVIDER and the API key)

cd web && npm install && cd ..
```

Example .env for Groq:

```
LLM_PROVIDER=groq
GROQ_API_KEY=<your key>
```

A.4 How to Run the System

1. Double-click start.bat in the project folder (or run it in a terminal). It opens one window for the backend (uvicorn lc.api:app, port 8000) and one for the web interface (port 5173), then opens the browser.
2. Open the Ingest page and upload one or more PDF or text files.
3. Open the Ask page and type a question. Click a citation to see its proof.
4. Close the two windows to stop the system.

The first start downloads the local models (embeddings, reranker and NLI model), which takes several minutes; later starts are fast.

A.5 API Endpoints

Table A.2 REST endpoints of the backend

Method	Path	Purpose
POST	/ingest	Upload a .txt or .pdf file: chunk, embed and index it
DELETE	/documents/{source_id}	Remove every chunk of one document
GET	/retrieve	Dense, keyword and hybrid (and optionally reranked) results for a query
POST	/query	Full pipeline: answer, VCS, decision and proof
GET	/stats	Number of documents and chunks indexed
GET	/embedding-map	2D PCA coordinates of every chunk
GET	/evaluation	Logged experiments with descriptions and headline numbers
GET / POST / DELETE	/chats, /chats/{id}, /chats/{id}/messages	Saved conversations
GET	/health	Liveness check

Sample request and response for POST /query (abridged; taken from the end-to-end test):

```
Request:
{ "query": "If the owner of my flat throws me out by force
            without permission, how much money am I owed?",
  "k": 12, "rerank": true }

Response:
{ "answer_mode": "verified", "decision": "ANSWER", "vcs": 1.0,
  "answer_text": "The tenant is owed compensation equal to
       six months' rent. [urban_tenancy_act_2019::s7:a]",
  "proof": { "claims": [ {
      "quoted_span": "A landlord found to have carried out an
           illegal eviction shall pay the tenant compensation
           equal to six months' rent.",
      "verdicts": {
        "v1_citation":   { "passed": true },
        "v2_entailment": { "verdict": "ENTAILS", "score": 0.99 },
        "v3_fidelity":   { "fidelity": 1.0 } } } ] } }
```

A.6 Troubleshooting

Table A.3 Common problems and solutions

Problem	Cause	Solution
“Could not reach the Legal AI backend”	Backend not running, or on another port	Run start.bat, or start uvicorn lc.api:app (port 8000)
HTTP 409 on upload	The document is already indexed	Use Replace on the Ingest page, or delete it first
HTTP 503 on a question	No API key, or the LLM provider is unavailable	Set the key in .env and restart the backend
Home page shows 0 documents	The LangChain backend has its own index	Upload the documents on the Ingest page
Many answers abstain	Strict NLI verdicts on correct claims	Open the proof to see which check failed; see Section 6.4
First question is slow	Models loading for the first time	Wait for the first load; later questions are faster

A.7 Security and Ethical Notes

- Never commit the .env file; it holds API keys. The repository ignores it by default.
- Uploaded documents stay in the local index, but questions and retrieved passages are sent to the LLM provider. Do not upload confidential documents unless the provider's terms allow it.
- The system is a research prototype and does not give legal advice. A verified answer means the cited passage supports the claim, not that the legal conclusion is complete.

A.8 Quick Run Checklist

1. Clone the repository and create the .venv-lc environment.
2. Install the Python and web dependencies.
3. Create .env with the LLM provider and key.
4. Run start.bat, upload documents on the Ingest page, then ask questions on the Ask page.

APPENDIX B – KEY CODE

The key code excerpts are given in Section 5.4: the citation-forcing system prompt, the VCSConfig dataclass that holds the weights, threshold and layer switches of the

verification chain, the LanceDB hybrid search, and the summary-augmented chunker. The full implementation of every module listed in Table 4.2 is in the GitHub repository (Appendix A.1), organised so that each pipeline stage can be read on its own.